centro de tecnología biomédica

UNIVERSIDAD POLITÉCNICA DE MADRID

medal

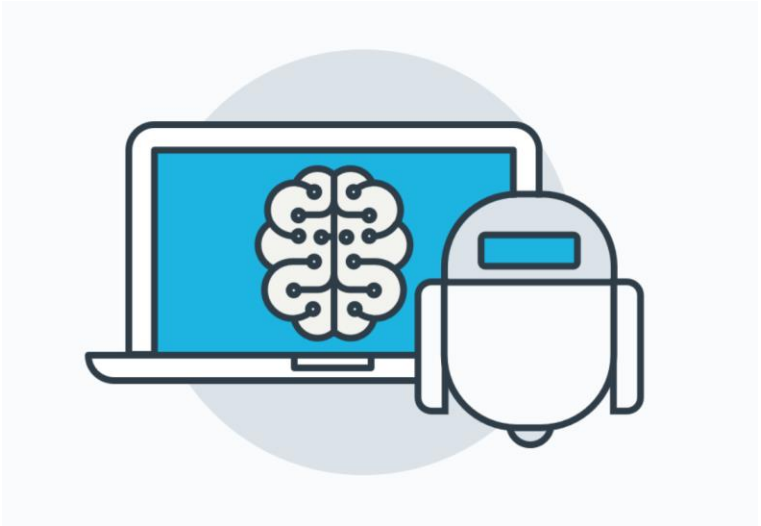
Computer Science for clinicians

Álvaro García Barragán
Center *for Biomedical Technology, Technical University, Madrid*

4^{th} December, 2023

Index

1. What is computer science?
2. Artificial Intelligent (AI)
3. Natural Language Processing (NLP)
4. Use cases in medicine
5. Conclusion



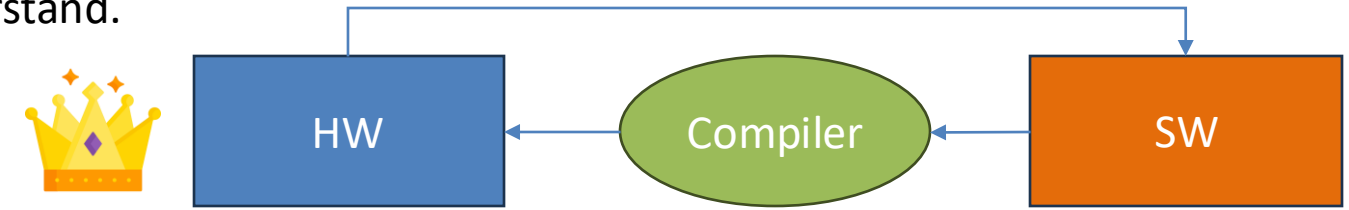
Computer Science

The software can be programmed in many languages; all that is needed is a "translator" who knows how to convert it into hardware

- Computers are used to **do specific task that can resolve humans** but very fast. (and other tasks that we cannot do).
- Can sort, search, do maths, visualize, communicate and much more with the information.
- The bases of informatic it's with very simple task as addition, subtraction, storing, and loading data, be able to do very complex task as serving a web or run a videogame.
- Everything works because this three concepts: **Software**, **Hardware**, **Compiler/Interpreter** ("translator").

Computer Science

- **Hardware**: physic component where the software its run.
- **Software**: set of instructions and scripts (*program*), can be written in different languages (*programming languages*), have algorithms. It's like the chef in a kitchen: the software tells the computer how to prepare the dishes.
    - There are different types of software, such as the operating system (which runs the computer), programs for editing photos, surfing the Internet, or playing video games. Each has a specific job.
- **Compiler/Interpreter**: It's like the communication between HW and SW. Can translate the high-level instruction to basic instruction that the computer can understand.



Programming Languages

What types or data are possible?

- **Primitives**
  - Int: 12
  - Double: 3.14
  - Character: ‘a’
- **Complex (Abstract Data Types)**
  - String (text): “Hola Mundo”
  - Tuple: (12, “Hola”)
  - List: [12, “Hola”]
  - Dictionary: {“Nota-Alvaro”:9, “Nota-Ines”: 10}
  - Objects

What can I do with the data?

- **Decisions** (jumps in the code)
  - Conditional jump (if, else), depends on a condition in the data.
  - Unconditional (break, return, exit), no depends in the data.
- **Loops**
  - Loops for search (while)
  - Loop to go though (for)

Programming Languages

Try to program more close to real life

Python examples:

Imperative

```python
persona = { "name": "Ines", "age": 24}


def what_is_your_name(persona):
    print("My name is: ", persona["name"])


what_is_your_name(persona) # Out: My name is:  Ines
```

Functional programming: functions, variables

Object-oriented

```python
class Persona:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def what_is_your_name(self):
        print("My name is ", self.name)

persona = Persona("Ines", 24)

persona.what_is_your_name() # Out: My name is  Ines
```

Objects, methods, attributes

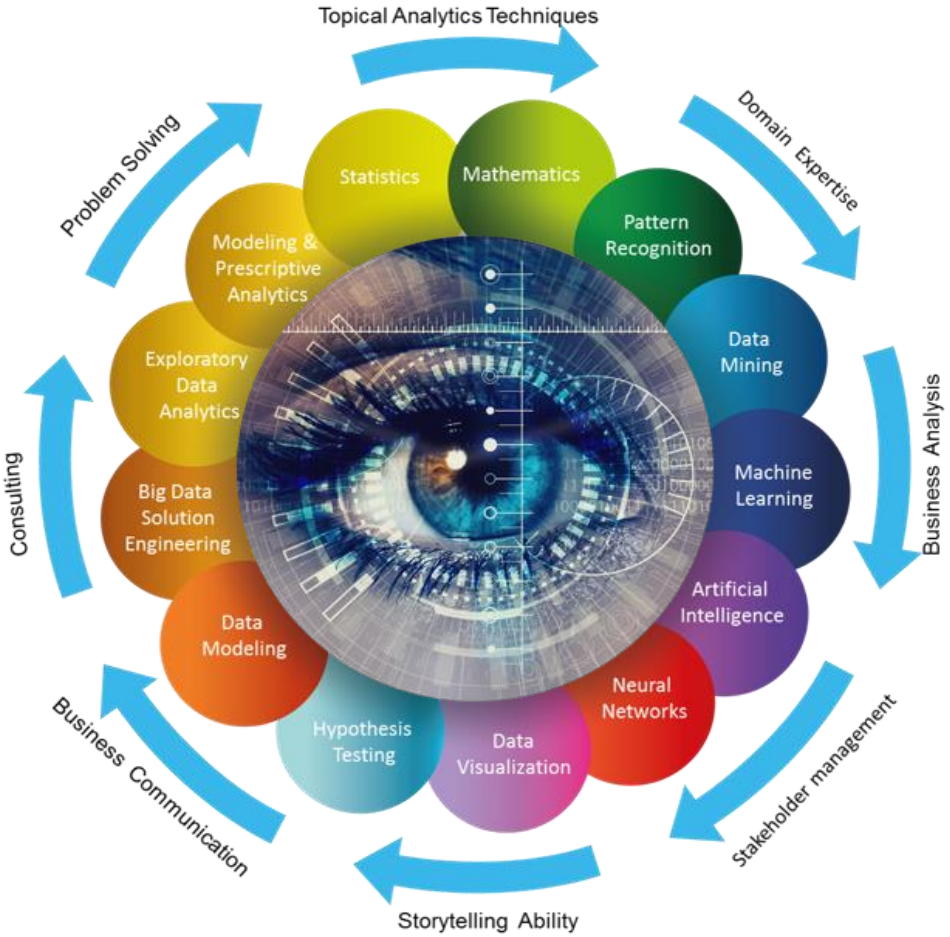
Programming Languages

Each Language have its own translator

- **Python (object-oriented, imperative):**
  - Simplicity and versatility, but more inefficient for do new algorithms.
  - Allow training very complex AI algorithms in easy way.
  - Well-known in Data Science field (pandas, sklearn or tensorflow).
- **Java (object-oriented):**
  - Robustness and security.
  - It's commonly used for mobile apps and complex business systems.
  - Very verbose, “A lot of code to declare data”.
- **C (imperative):**
  - Performance and control (less high level, more near the machine).
  - Is a powerful tool favored by experts for precise control over computer systems.
  - It's used to build core components of operating systems and other software.

The are languages more specific to a domain

Artificial Inteligencie



What is IA?

"The theory and development of computer systems to carry out tasks that normally require human intelligence, such as visual perception, speech recognition, decision-making, or language translation" - Oxford Living Dictionary **[Dartmouth Conference, 1956]**

Tasks that require human intelligent

- Prediction of outcomes and pattern recognition based on historical data
- Recognition of anomalous events and decision-making
- Interpretation of visual information
- Language understanding and engagement in conversations
- Extraction of information from sources to gain insights

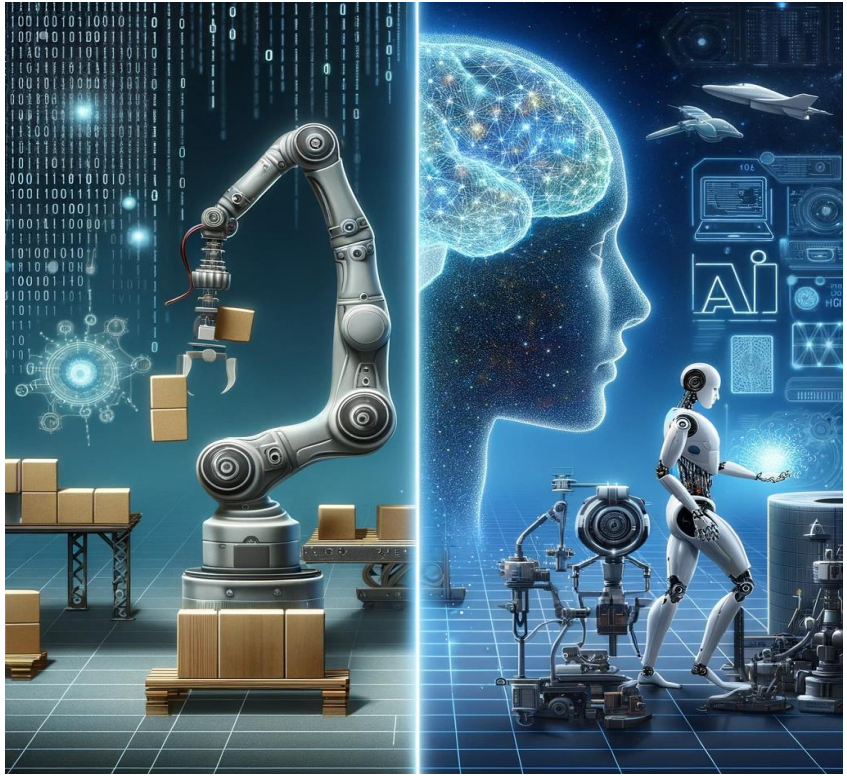
Common workloads in AI

Not every model of NLP is Machine Learning

 Machine Learning	Data-driven predictive modelling: the foundation of AI
 Anomaly Detection	Systems that detect unusual patterns or events, allowing preventive action to be taken
 Computer vision	Applications that interpret visual input from cameras, images or videos
 Natural Language Processing	Applications that can interpret written or spoken language, and participate in dialogues with human users

Classification of AI

Weak artificial intelligence, or narrow AI, specializes in **specific tasks**, simulating human cognition. It **lacks genuine understanding or consciousness** and operates within predetermined limits.



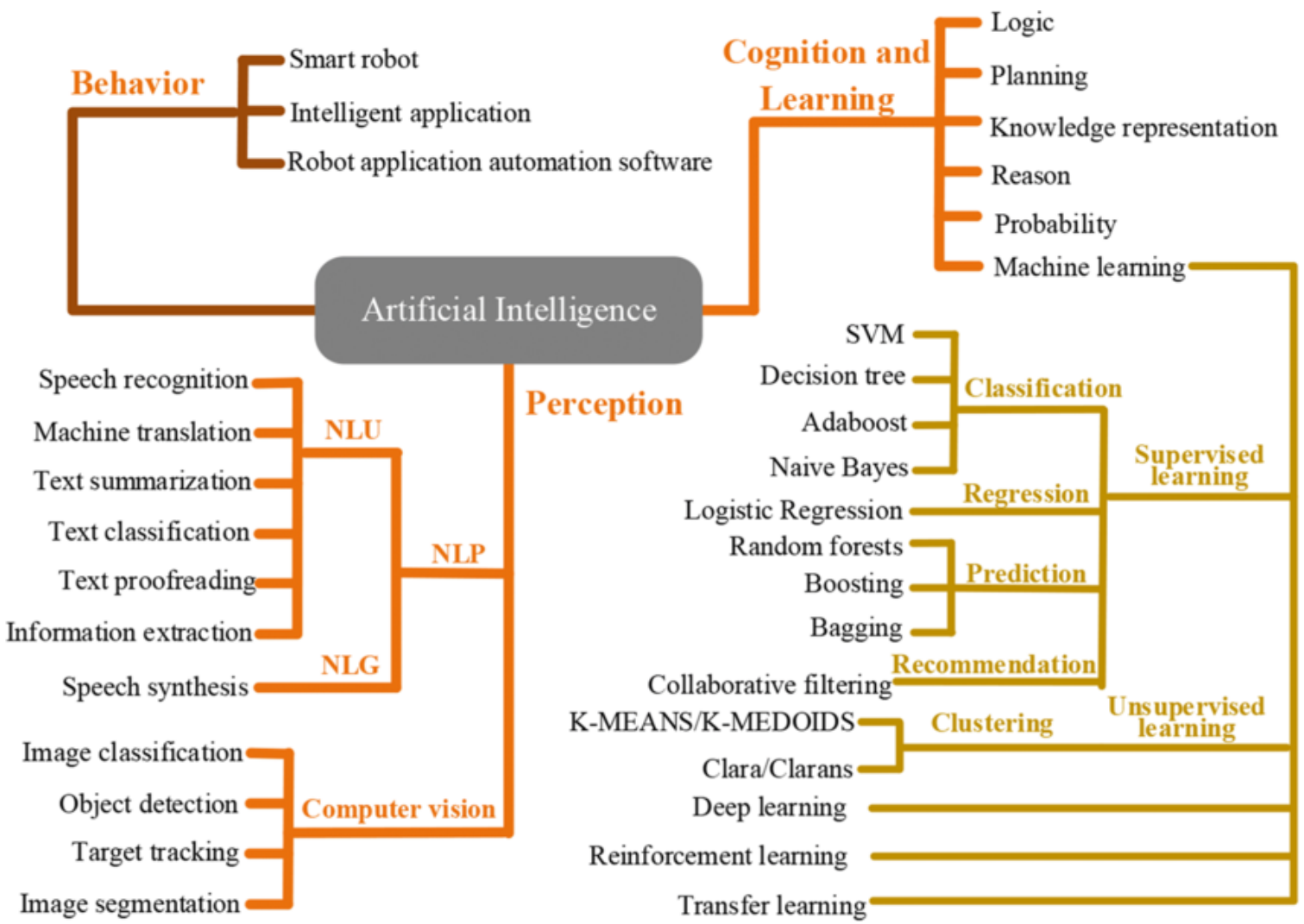
Strong artificial intelligence, or general AI, is a hypothetical system that **equals human cognition**. Endowed with self-awareness, adaptability, and understanding, it could perform any intellectual task that a human being can do

Classification of AI

Classification more “known”:

- **Symbolic AI**: is based on the use of formal languages and unambiguous processes to determine when a statement is true or false in each situation. This leads to propositional logics, first-order logical systems, and languages like Prolog. Ultimately, this leads to expert systems that emulate expert behavior based on symbolic reasoning.
- **Non-symbolic AI**: is based on the simulation of human cognitive processes such as perception, learning, and decision-making. Non-symbolic approaches include neural networks, genetic algorithms, fuzzy systems, and agent-based systems.

MORE CLASSIFICATIONS OF AI



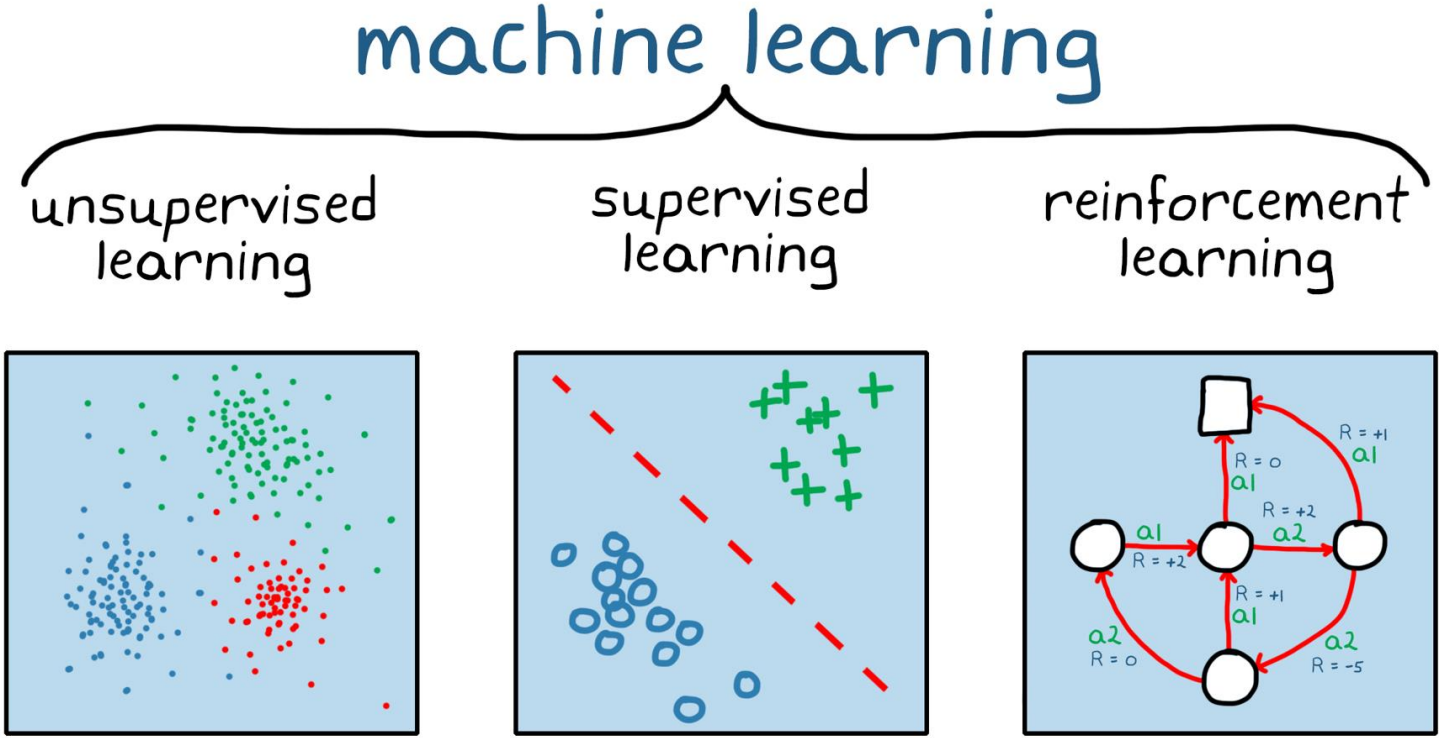
Example of AI Taxonomy

Wang, L., Liu, Z., Liu, A., & Tao, F. (2021). Artificial intelligence in product lifecycle management. The International Journal of Advanced Manufacturing Technology, 114, 771-796.

Machine Learning (ML)

- In **machine learning**, instead of programming rules and decisions directly into a computer system, algorithms are used that can analyze ***data***, identify patterns and learn from them.
  - **Training [Labelled] Data**: These are datasets that are used to teach an algorithm
  - **Model:** is the mathematical representation that the algorithm uses to make predictions.
  - **Learning from data:** The model adjusts its parameters or structure to make more accurate predictions as it is provided with more training data.
  - **Prediction or inference:** After training, the model makes predictions on new, unseen data or situations.
  - **Types of ML:** Supervised, No supervised, Reinforcement Learning.

Classification of ML



Classification of ML

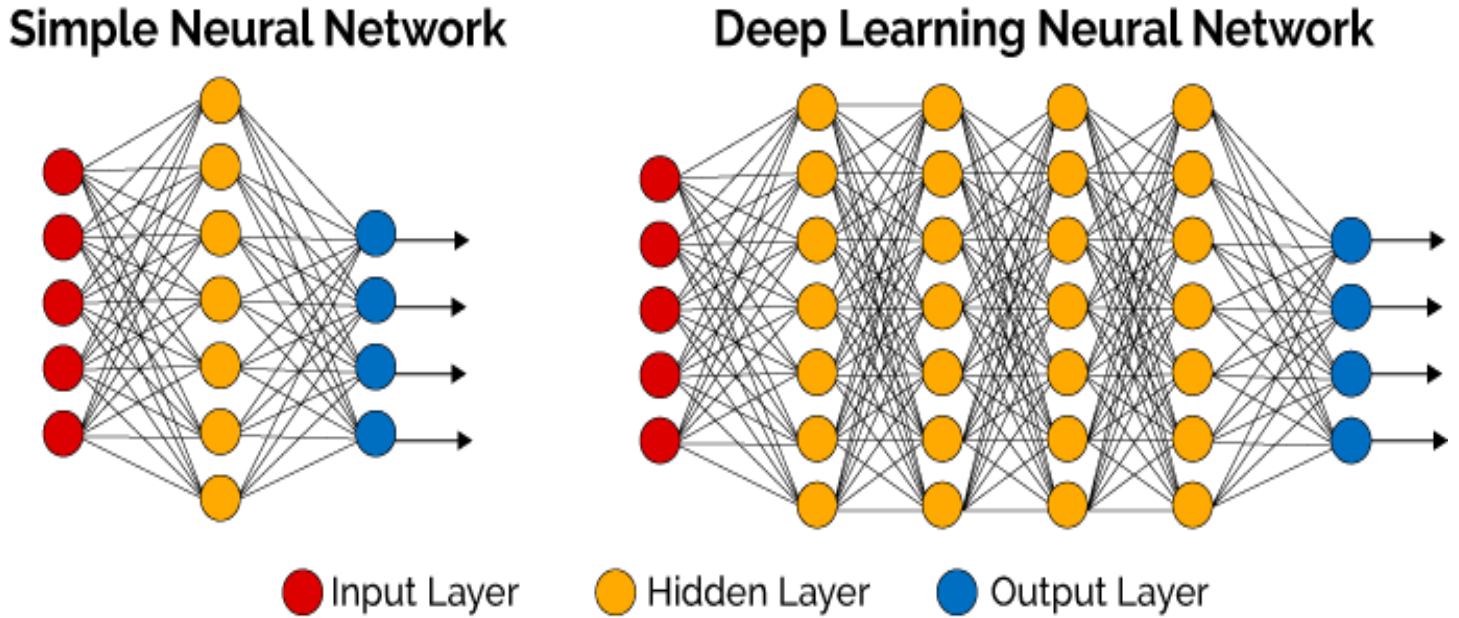
In a real project can be use together

- **Supervised Learning** (Labelled Data, train-test): predict the target value (output variable) based on known values of features variables (input variables). Model training with train-split and evaluate it with test-split. Main types of analytics problems:
  - *Regression Problems* (output is a number)
  - *Classification Problems* (output is a class)
- **Unsupervised Learning** (non labelled data): Cluster analysis or Association analysis.
- **Reinforcement Learning** (agent-environment-reward): agent exploring an environment, aiming to determine optimal actions or sequences without relying on examples of correct responses. Instead of labeled training data, the learning process involves a reward function that provides feedback on actions taken. The agent endeavors to maximize its reward through an iterative trial-and-error process, making reinforcement learning practical in situations where optimal actions are unknown, and supervised learning with labeled data may lead to suboptimal results.

ChatGPT: Is a mix of them (RLHF, SL (finetuning), UL (pretraining)

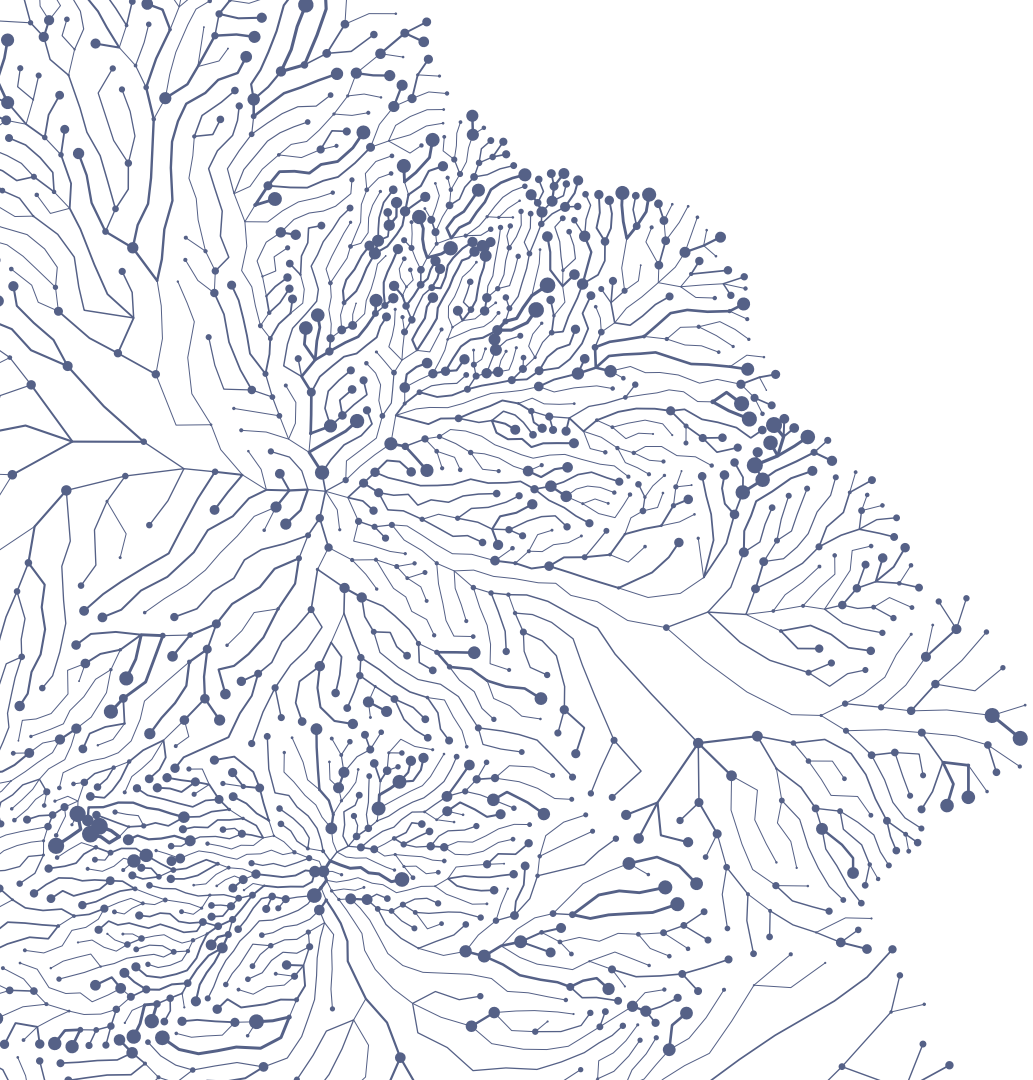
Deep Learning

A form of machine learning based on the same principles as the brain's neural networks.
[https://mlu-explain.github.io/neural-networks/]



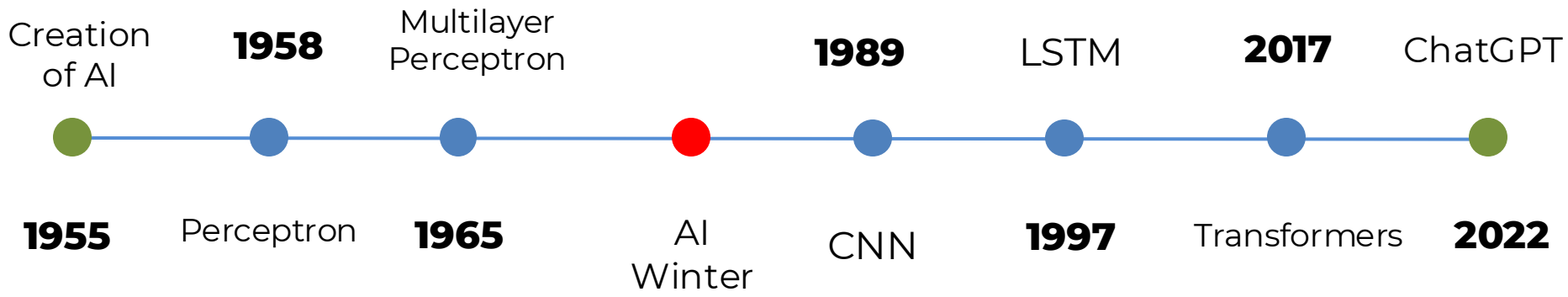
ML “best” models for “classifying” data

- **Structured Data** (Tabular Data):
  - XGBoost, LightGBM, Random Forest, SVM.
- **Unstructured Data**:
  - **Audio**: WaveNet, YAMNet.
  - **Video**:
    - 3D Convolutional Neural Networks (3D CNNs).
    - I3D (Inflated 3D CNN).
  - **Image**:
    - Convolutional Neural Networks (CNNs).
    - YOLO (You Only Look Once).
  - **Text**:
    - RNN, BiLSMT, LSTM, BayesianNetworks.
    - BERT (Bidirectional Encoder Representations from Transformers).
    - GPT (Generative Pre-trained Transformer).



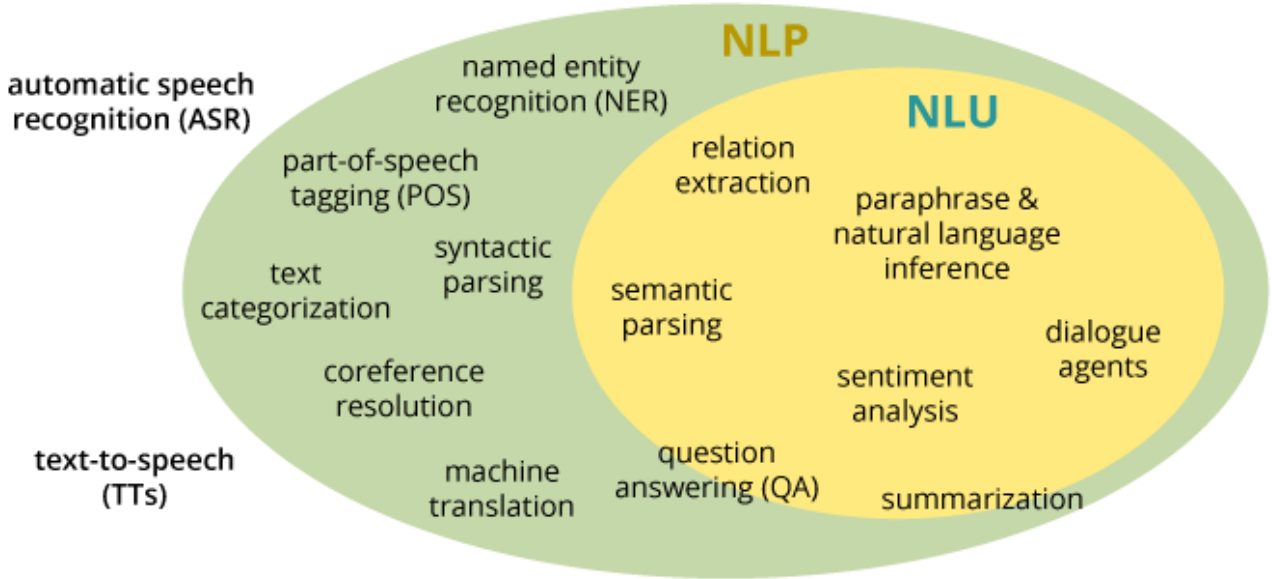
Natural Language Processing

NLP with ANN


Creation of AI
1955
1958
Perceptron
Multilayer Perceptron
1965
AI Winter
1989
CNN
LSTM
1997
2017
Transformers
ChatGPT
2022

NLP

Natural language processing (NLP) refers to the branch of AI concerned with giving computers the ability to understand text and spoken words in much the same way human beings can. [LINK]

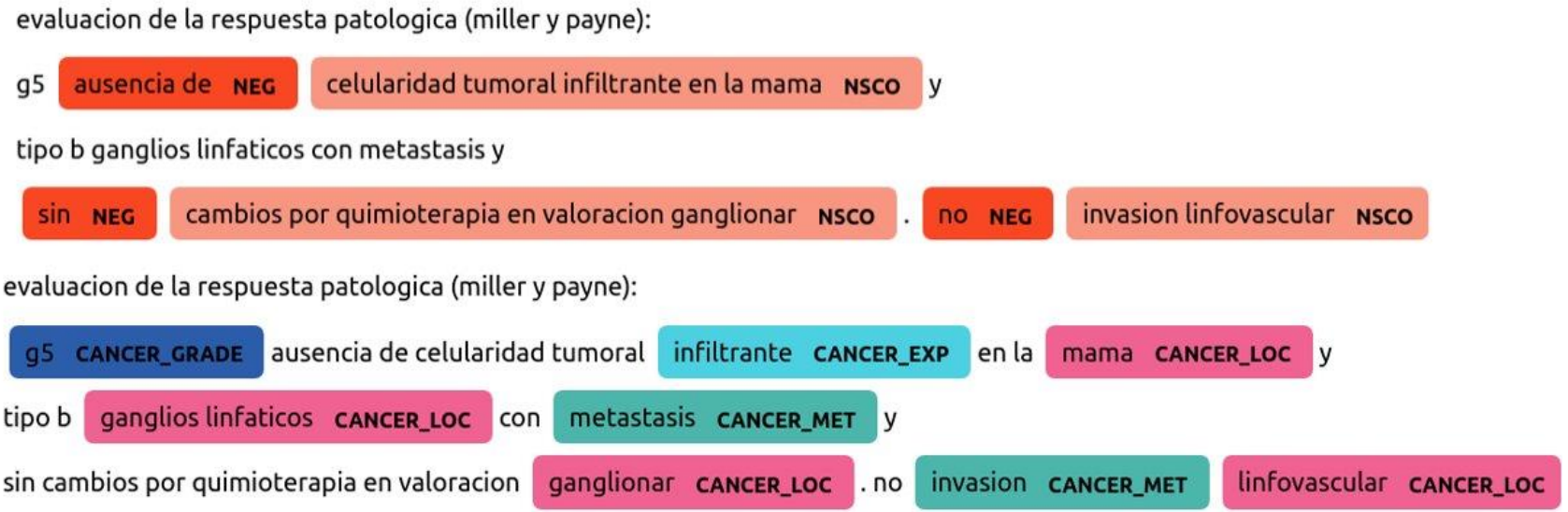


NLP basic tasks

- "The fundamental aim in the linguistic analysis of a language L is to separate the grammatical sequences which are the sentences of L from the ungrammatical sequences which are not sentences of L and to study the structure of the grammatical sequence" [**Syntactic Structures, 1957**]
- Syntactic analysis plays a crucial role in various tasks by evaluating how language conforms to grammatical laws. This involves applying grammatical rules to words to extract meaning through several techniques:
  - **Lemmatization**: Simplifying varied forms of a word to a single, base form for streamlined analysis.
  - **Stemming:** Trimming words to their root form.
  - **Morphological Segmentation:** Breaking down words into their smallest units of meaning, called morphemes.
  - **Word Segmentation:** Separating continuous text into individual, meaningful units.
  - **Parsing:** Conducting a detailed grammatical analysis of sentences.
  - **Part-of-Speech Tagging (POS):** Determining the grammatical role of each word in a sentence.
  - **Sentence Breaking:** Identifying and marking the boundaries of sentences in a continuous stream of text.

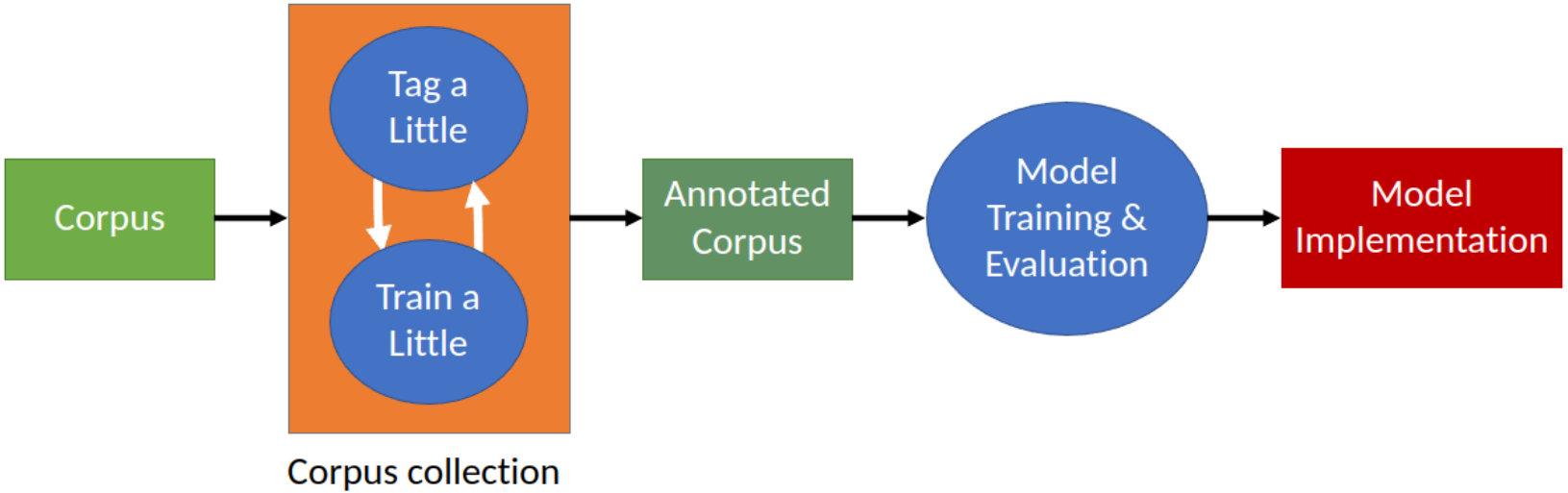
NER

- **Named Entity Recognition (token classification)** is a natural language processing task that involves identifying and classifying named entities, within a given text.
- In ML this models are trained with a labeled **corpus.**



Stages to obtain a NER model

- Generating a NER model in ML is an **iterative** and **incremental** process.
- The number of entities required for each tag depends on the architecture and data.
- Annotate a little then check is the model capable of predict it.



NER with spaCy

```
import spacy
from spacy import displacy

our_model = "models/lung-0.8/roberta/model-best"

nlp = spacy.load(our_model)
```

✓ 2.7s

```
doc = nlp("cancer de pulmon en paciente mayor 69 años")
print(doc.ents)
displacy.render(doc, style='ent')
```

✓ 0.0s

```
(cancer de pulmon, 69, años)
```

cancer de pulmon CANCER_CONCEPT en paciente mayor 69 QUANTITY años METRIC

spaCy

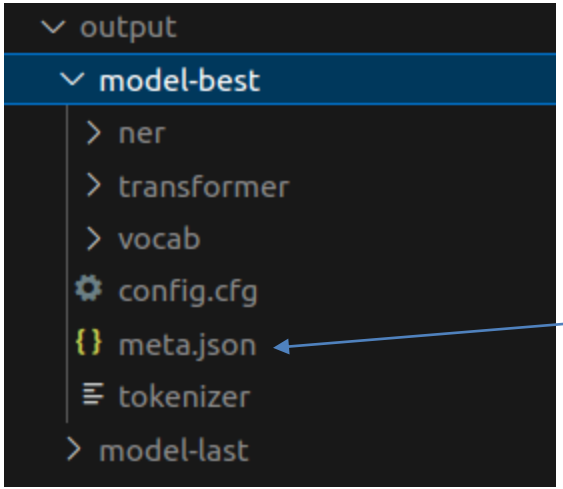
NER with spaCy

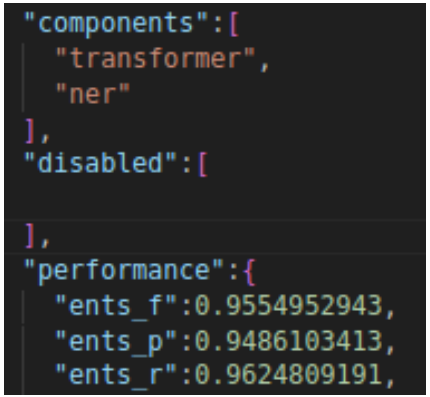
- Obtain a corpus and do EDA (Exploratory Data Analysis) for NER.
  - Number of tokens, punctuation marks, writing mistakes, ...
- Define Annotation guideline with experts in that field.
- Label the corpus (Prodigy tool).
  - The more data the better the corpus
  - If more than one annotator => Metrics of bias between them (IAA).
- Export the corpus in spacy format.
- Create config file for training.
- Train the models with spacy.
- See the evaluation metrics, generated in the output directory defined in config.

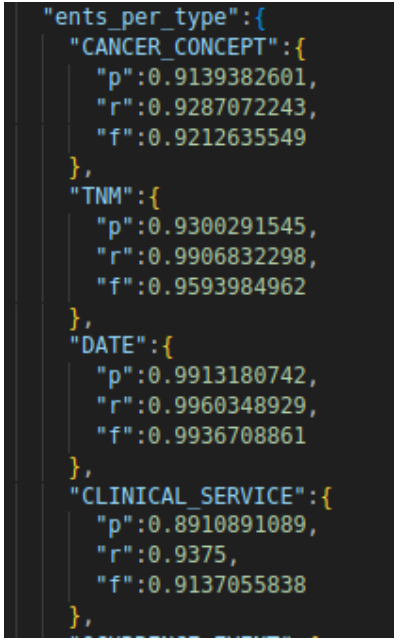
```
from spacy.cli.train import train

train("./config.cfg", overrides={"paths.train": "./train.spacy", "paths.dev": "./dev.spacy"})
```

NER with spaCy





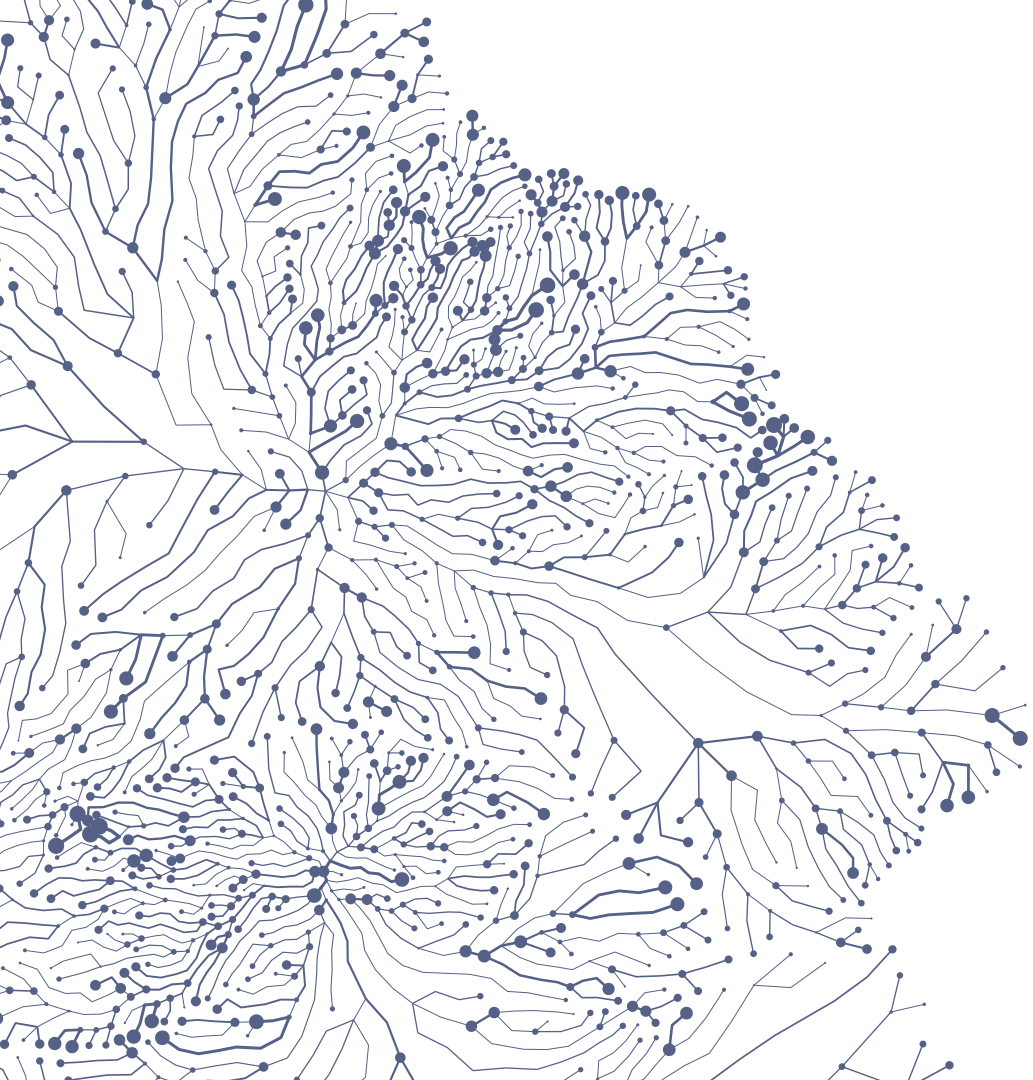


In NER, the F-score is the most used metric (accuracy not recommended).

NER evaluation metrics

The metrics should display the F-score for each entity because:

- **Class imbalance**: named entities are typically a small proportion of the dataset compared to unlabeled tokens. If a model predicts all tokens as non-entity, it could still achieve high accuracy due to the class imbalance, but its ability to identify actual entities may be deficient.
- **Importance of entities**: some entities may be more critical or important than others. For instance, in medical entity identification, correctly identifying diseases may be more crucial than identifying other categories. Accuracy does not consider this weighting of importance.
- **Emphasis on recall**: In NER, recall is often valued more than precision. It is crucial to capture as many real entities as possible, and precision alone does not adequately reflect this.
- **True negatives are not considered**, in NER not tagging word that don't have label don't say anything about the model. (for this reason, accuracy is not used)



How to input text into an Artificial Neural Network (ANN)?

Steps

Make understandable to ANN (have a maximum number of tokens):

Raw text => Tokens => Embedding => Model

Make understandable for human read (end when especial token or max number of tokens is reach):

Model => Embedding Edited => Tokens => Raw Text

Tokens

Computers don't understand words only numbers

medal

¿Cómo se pueden meter palabras en un Red Neuronal?
Tokenización + Embeddings

Token - Unidad mínima de codificación

División por palabras

Divi sión por sub pa la bras

D i v i s i ó n p o r c a r a c t e r e s

BTE (Byte pair encoding) (GPT, LLama)

In : Aaabdaaabac

Y=ab

Z=aa

X=ZY

Out : XdXac

Types of Coding

- **Integer-encoding**: assigns unique numerical identifiers to words.
- **One-hote-encoding**: represents them as binary vectors.
- **Word-embeddings**: provide compact vector representations in a continuous space, capturing semantic relationships to enable a better comprehension.

Necesaria conversión de los tokens a números (codificación y decodificación)

Types of Coding

INTEGER ENCODING

Perro = 1

Gato = 2

Mapache = 30

ONE-HOTE-ENCONDING

Perro = [1 0 0]

Gato = [0 1 0]

Mapache = [0 0 1]

Integer-encoding: ¿La palabra mapache dista de perro en 29 unidades?

One-hote-encoding: vocabulario de 10000 palabras necesitamos 10000 vectores de 10000 componentes ¿Cada palabra dista lo mismo del resto?

Especial Tokens BERT

- **[CLS]**: placed at the beginning of each input sequence. It is mainly used in classification tasks.
- **[SEP]**: used to separate different sentences in the same entry.
- **[MASK]**: This token is central to BERT training in the Masked Language Modeling (MLM) task. It is used to replace random words in the sentence during training. The goal of the model is to correctly predict the original word that has been masked.
- **[PAD]**: used to fill in input sequences until they are all the same length so that they can be processed in batches by the model. BERT requires that all input sequences have the same length. (all the input/output have the same size)

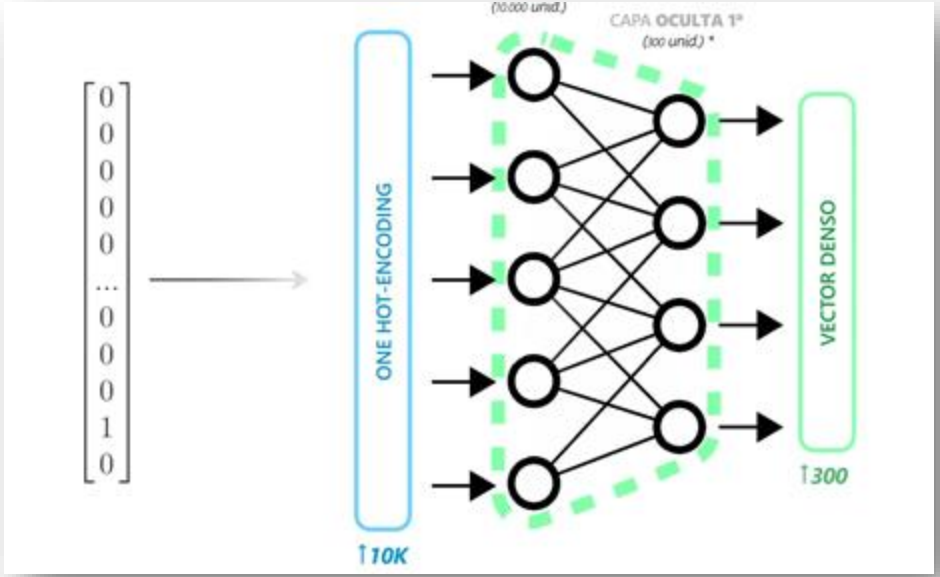
Especial Tokens GPT

- **[EOS]:** The End Of Sequence token is used to indicate the end of a text sequence. This token is essential in text generation to determine when the model should stop generating more text.
- **[BOS]:** Some versions of GPT may include a Beginning Of Sequence token, although its use is not as common as the [EOS] token. It is used to mark the beginning of a new text sequence.
- **[UNK]**: The Unknown Word token may appear in situations where the model encounters a word that is not in its vocabulary. However, given the large vocabulary of GPT, the occurrence of this token is relatively rare.
- **[PAD]**: As in BERT, the Padding token is used to match the length of input sequences, although in text generation practice, this token has a less prominent role than in models such as BERT. (the input size differ from the output size)

Embeddings

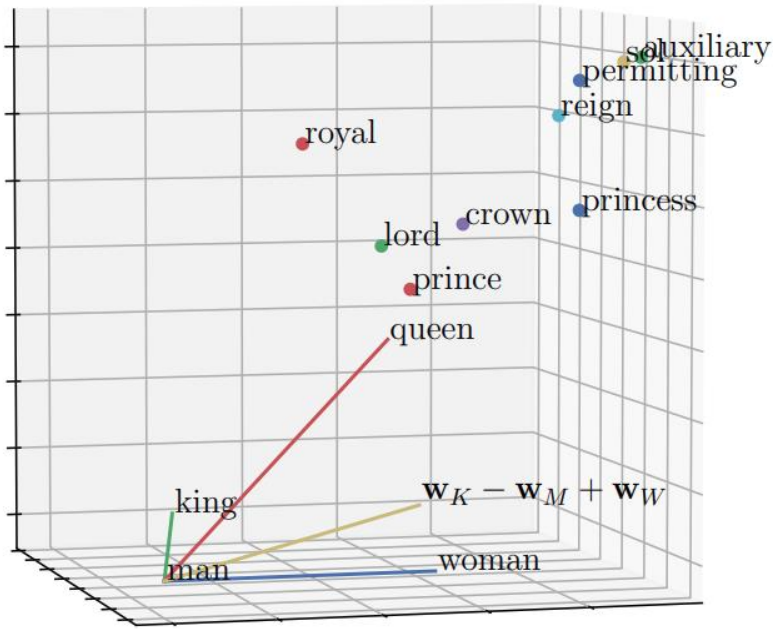
Convertir representación one-hote-encoding a representación más compacta

Solution: **word embeddings** expressing the semantic relationship of the language.



Embeddings

- **[Google 2013]** -> word2vec (trained model that encodes tokens to vectors).
- Vector Space Operation. Similar words have proximate vectors in space.

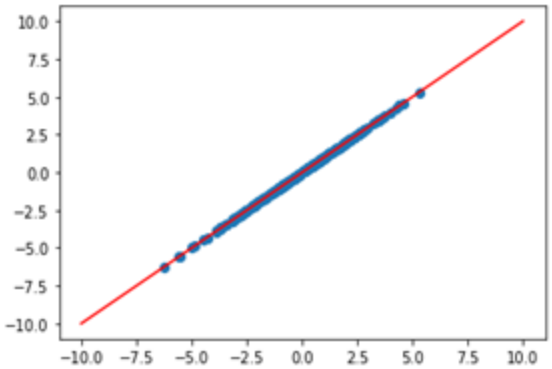


Dimensionality Reduction (PCA) 300->3, http://projector.tensorflow.org/

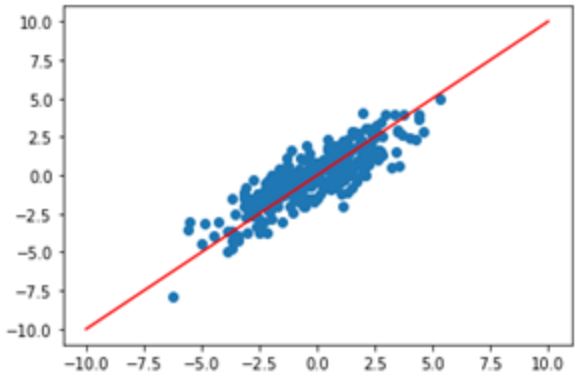
Embeddings similarity

- Que similitudes hay entre perro y gato (animales, mascotas y sustantivos)
- Similitud entre palabras en vectores de 300 elementos , grafico de dispersión

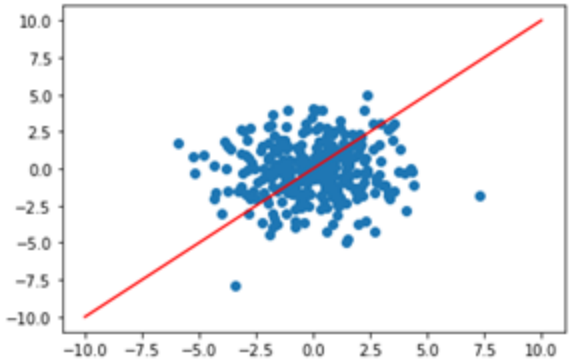
```
Word1:  gato
Word2:  gato
Similaridad: 1.0
```

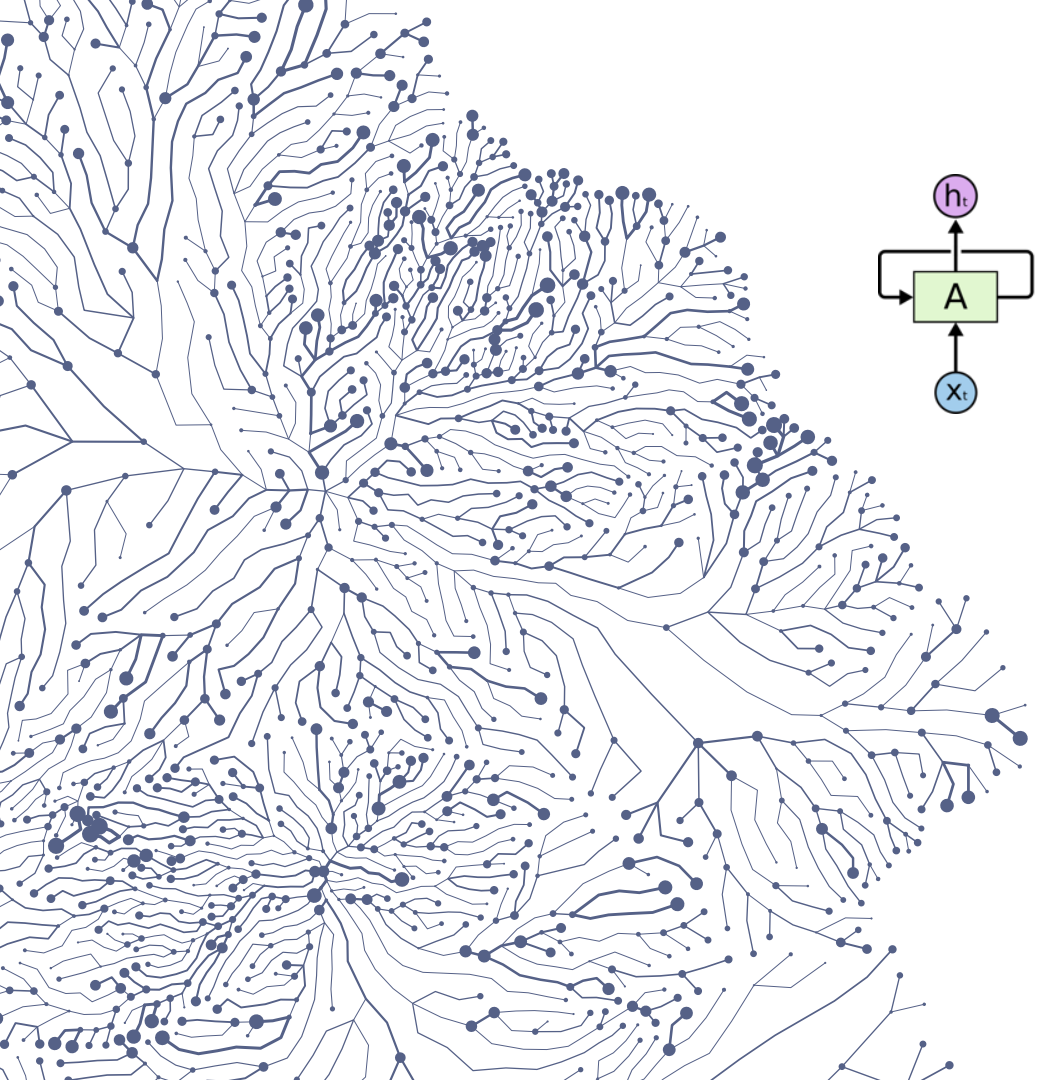


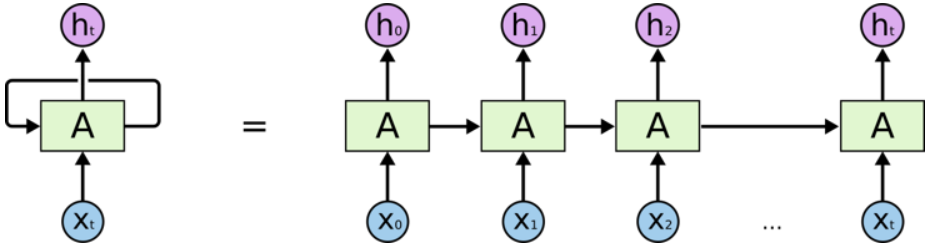

```
Word1:  gato
Word2:  perro
Similaridad: 0.8487285375595093
```




```
Word1:  hola
Word2:  perro
Similaridad: 0.05708715319633484
```



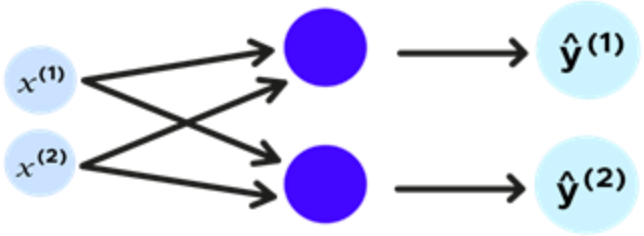




RNNs

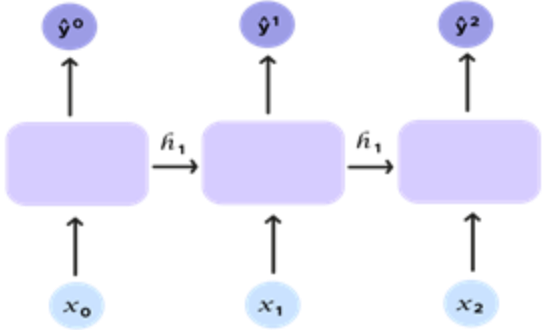
Multilayer vs RNN

Multilayer



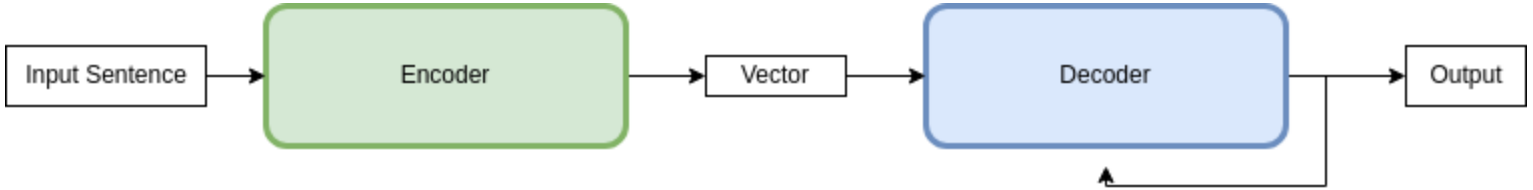
Each token is independent

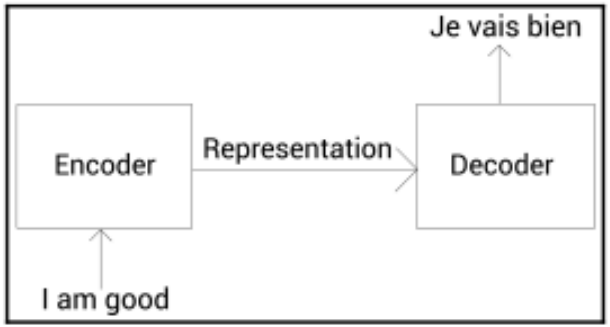
RNN



Each token depends on the previous ones

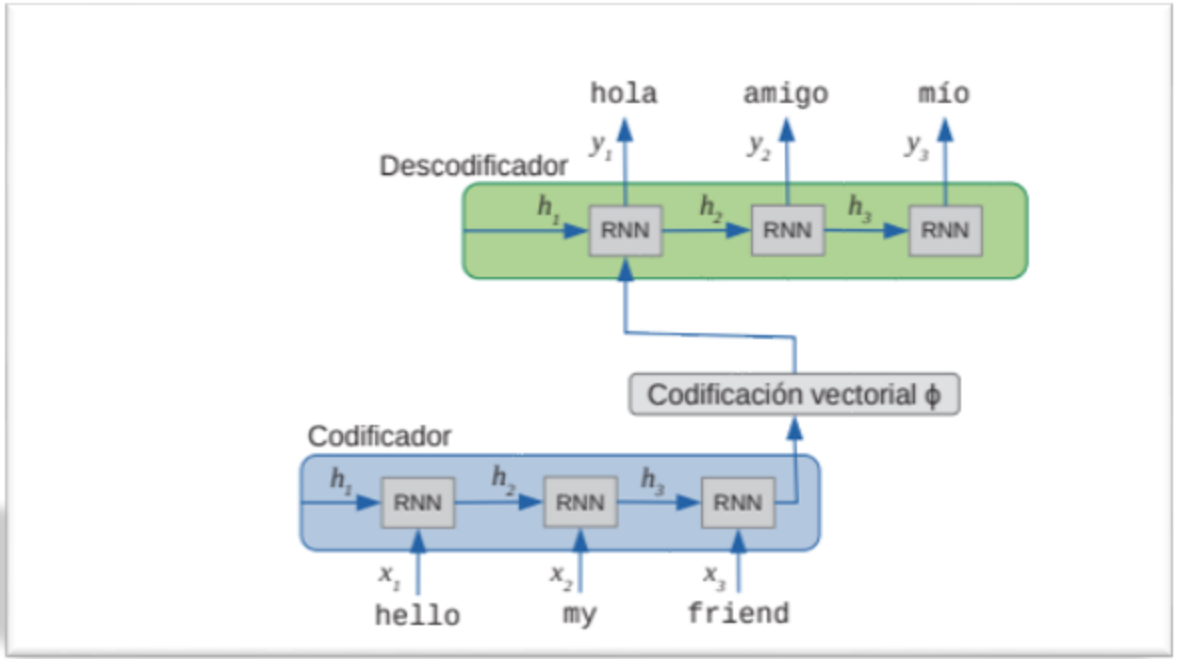
Encoder-decoder models: *Seq2Seq*





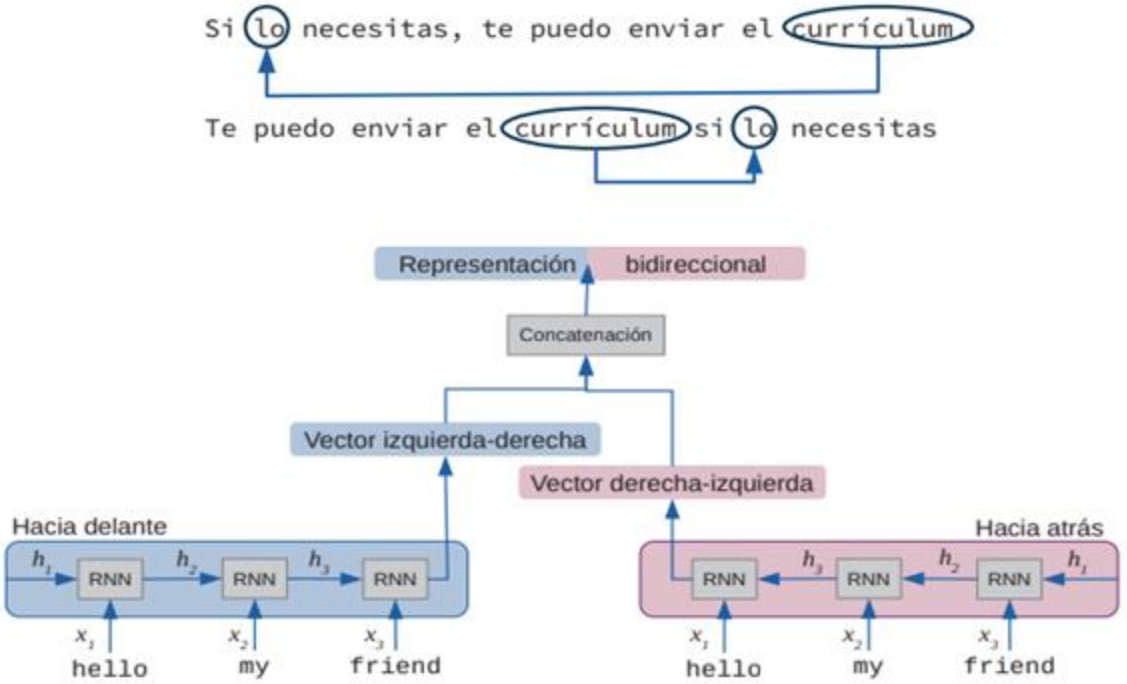
Embeddings similarity

Ejemplo
modelo Seq2Seq de
traducción con RNN

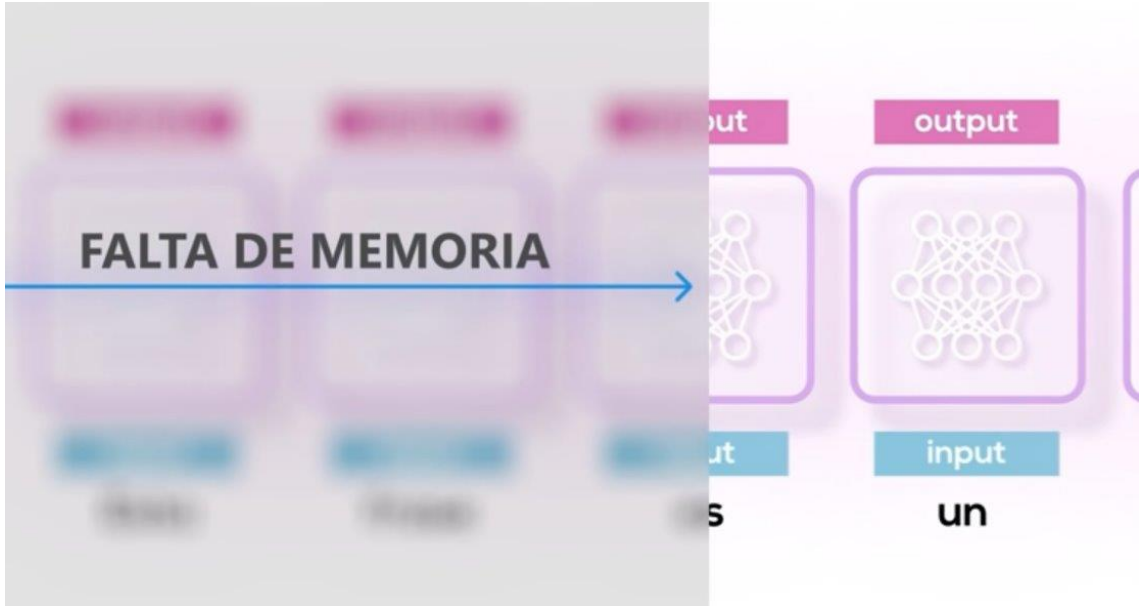


Recursive bidirectional models

Necesario bidireccionalidad para entender el texto

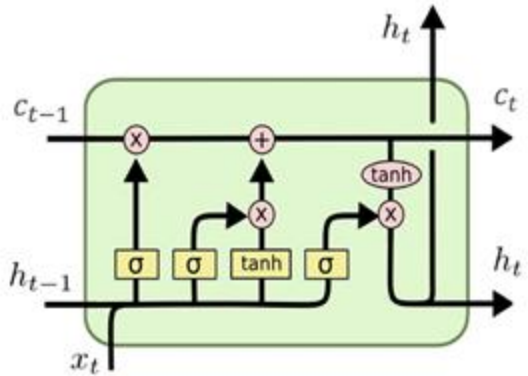


Recursive models problem



LSTM

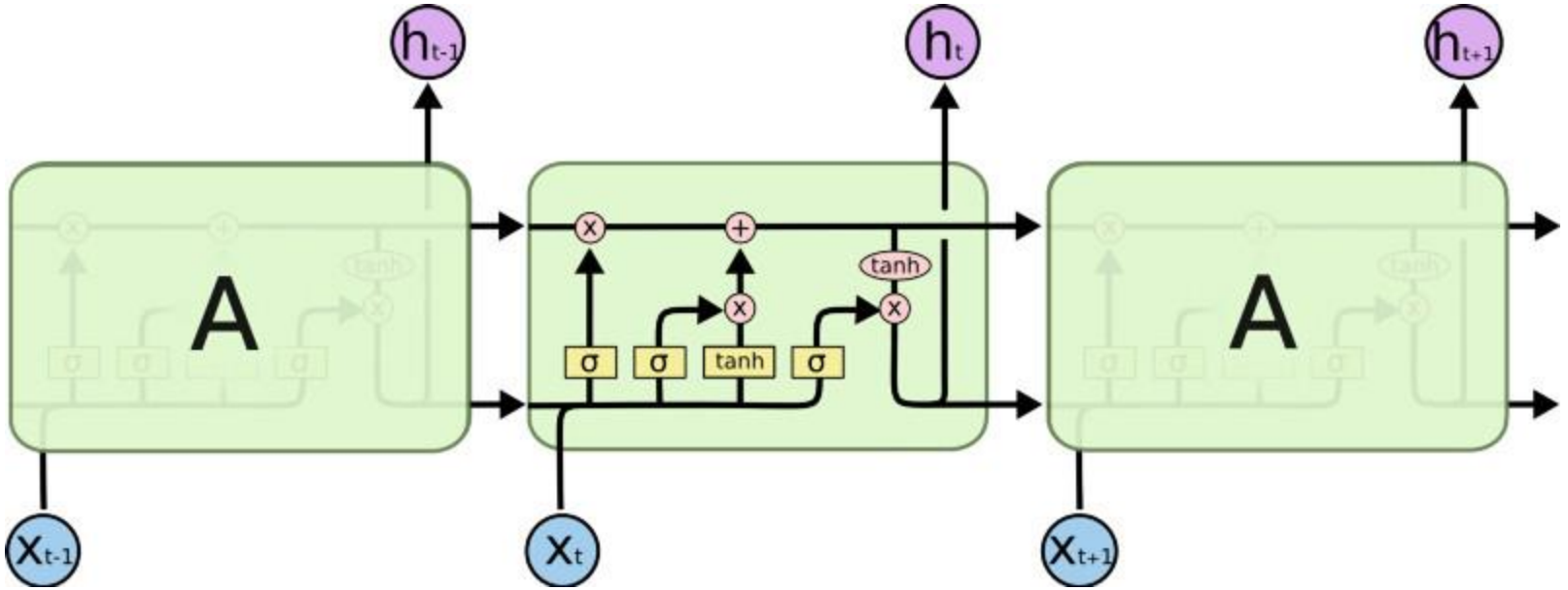
LSTM - Long Short-Term Memory Networks



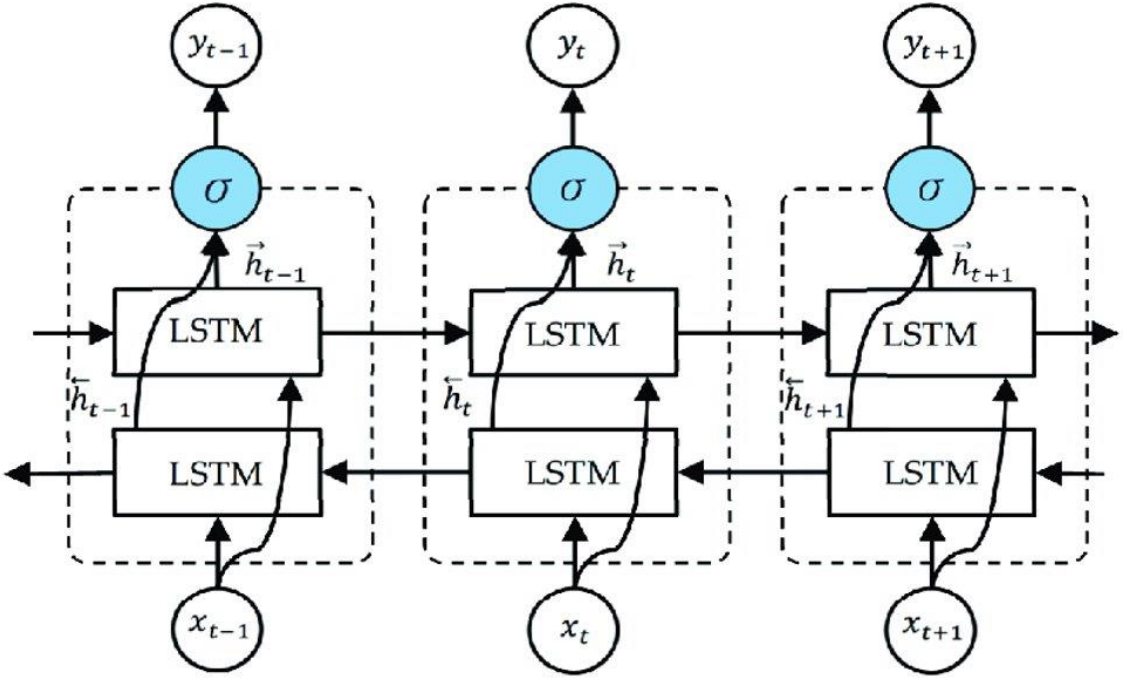
Pasamos de dos entradas a tres

- Token actual (xt)
- Salida celda anterior (ht-1)
- Memoria (ct-1)

LSTM



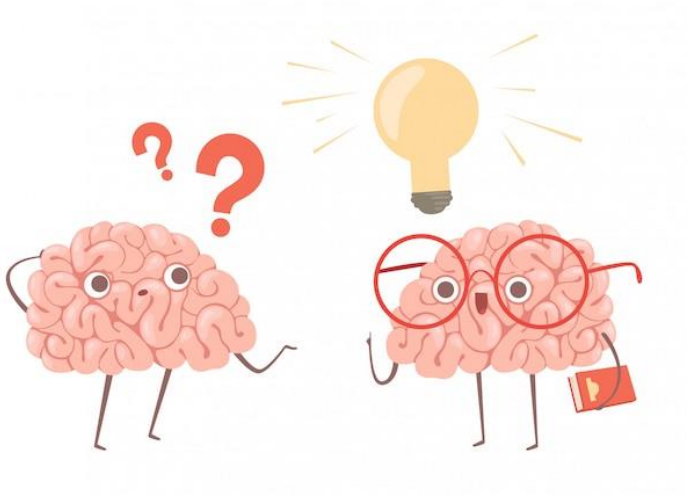
BiLSTM


y_{t-1}
y_t
y_{t+1}
σ
σ
σ
$\vec{h}_{t-1}$
$\vec{h}_t$
$\vec{h}_{t+1}$
LSTM
LSTM
LSTM
$\overleftarrow{h}_{t-1}$
$\overleftarrow{h}_t$
$\overleftarrow{h}_{t+1}$
LSTM
LSTM
LSTM
x_{t-1}
x_t
x_{t+1}

RNN problems

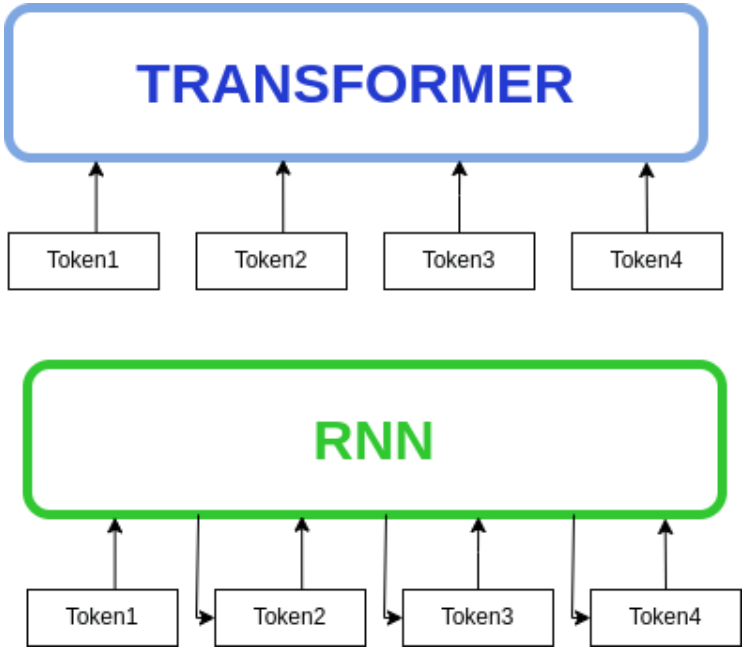
Efficiency and scalability problem:

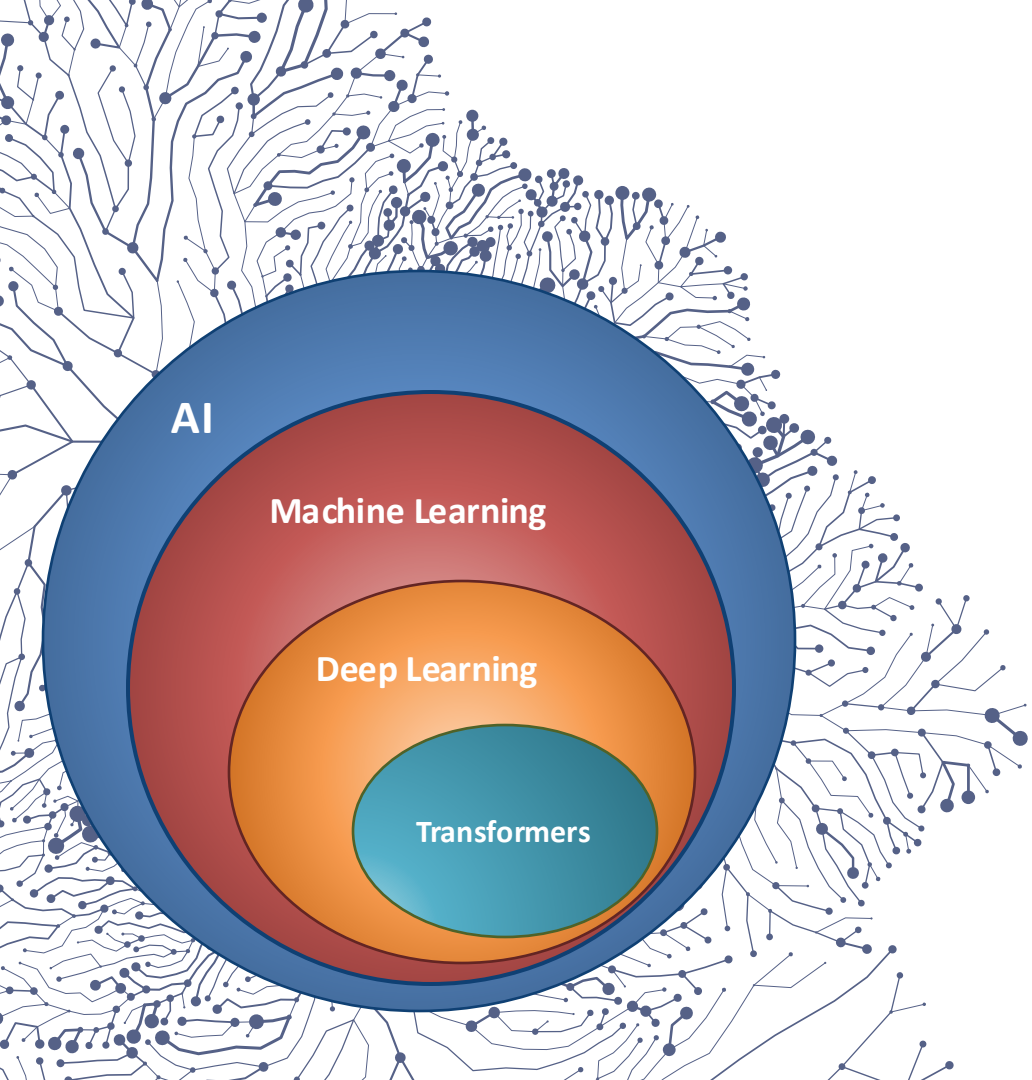
- They are computationally very expensive to train for very long sequences.
- Slow training, does not allow parallelization of learning.



Transformers vs RNN

- In RNN, each word must wait until the previous words have been process.
- RNNs have a problem with forgetfulness (gradient descent).
- On the other hand, transformers process the sentence at once and therefore parallelize the process, being able to process a greater amount of data and being more efficient in training.
- How is it achieved?





Transformers

Attention

- *Attention is all you need* [2017]: La atención es un mecanismo clave que permite al modelo centrarse en partes relevantes de la entrada y producir una salida contextualizada (Simulando la concentración humana).

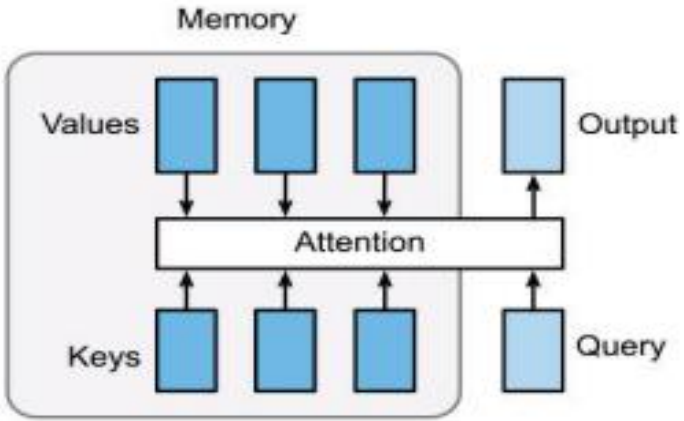
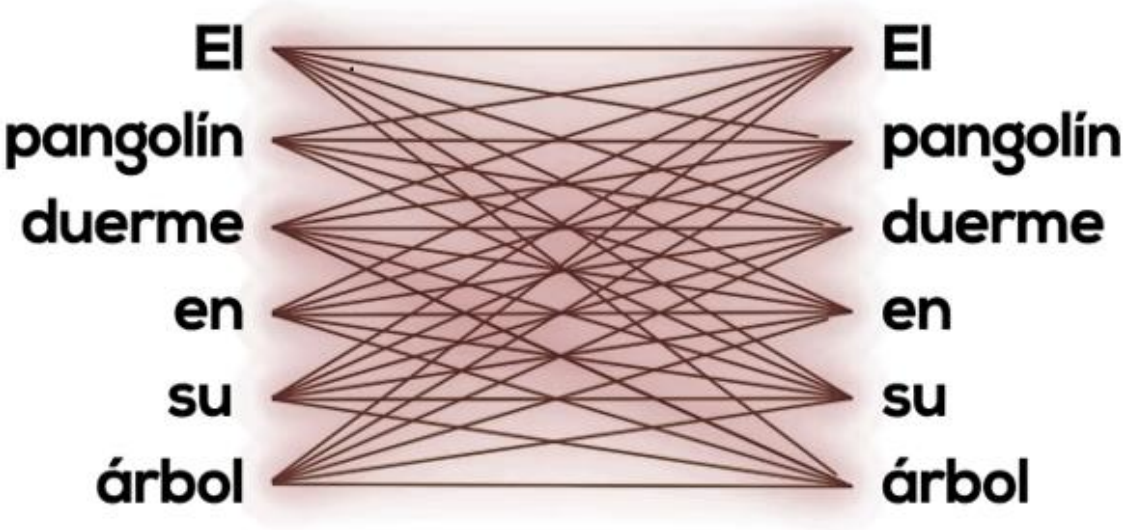


Figura 2.10: (Zhang et al., 2020a)

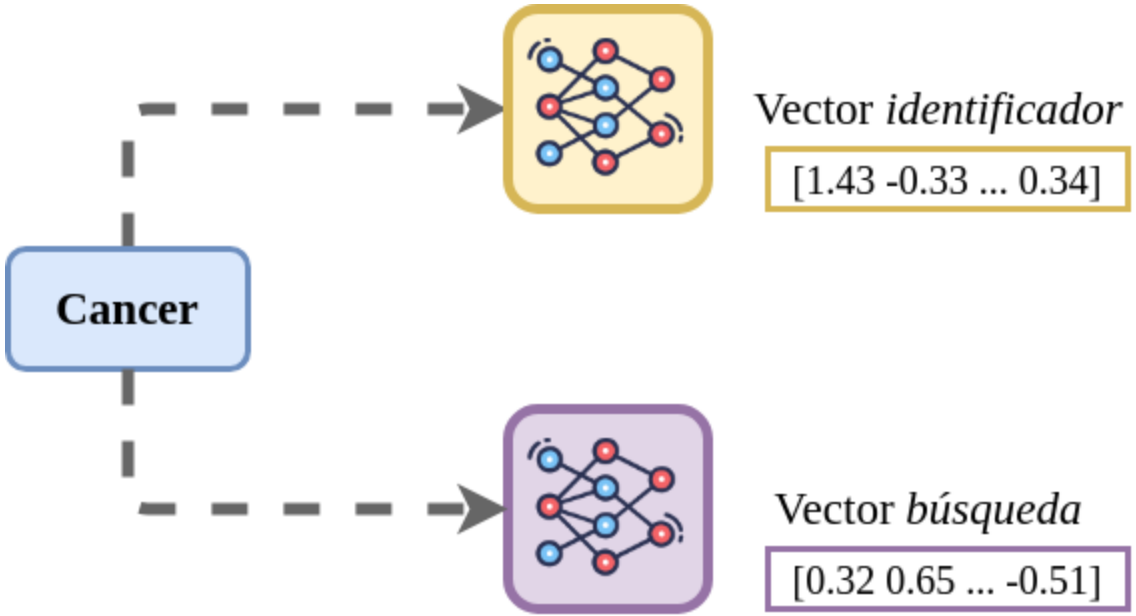
Self-Attention



La red aprende la relación de todas las palabras con todas las palabras.

Existen palabras con mayor relación contextual como pangolín y árbol (naturaleza).

Self-Attention



Vector identificador (key): se utiliza para determinar qué posiciones de la entrada son importantes para una posición dada.

Vector búsqueda (query): se utiliza para determinar qué información es relevante para la posición actual.

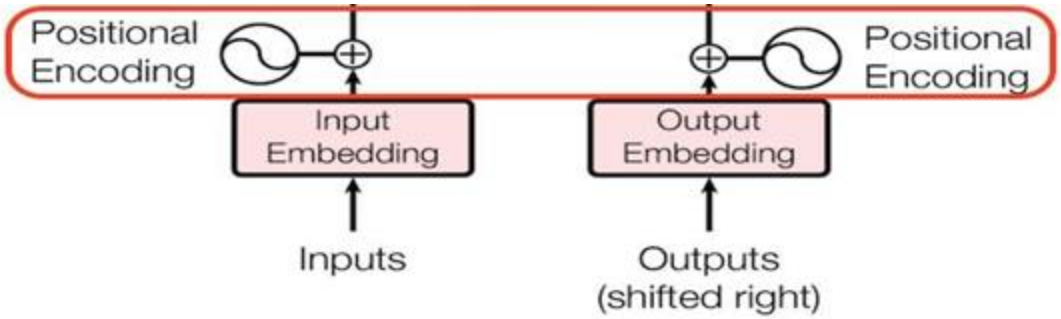
Self-Attention

Vector Query (Cerradura)
Vector Key (Llave)

Atención = V_Query x V_Key (Producto escalar)

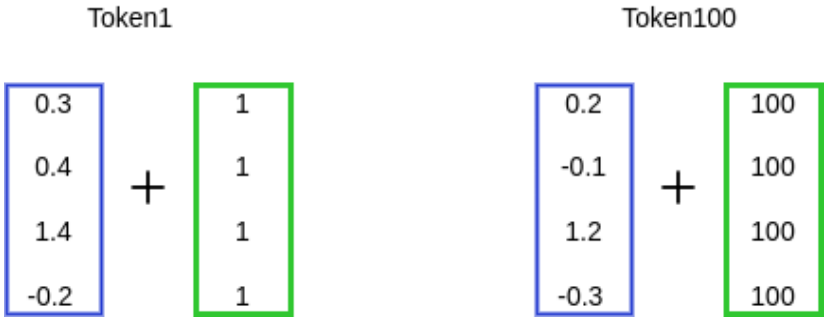
Position Encoding Problem

- How can transformers, in a parallel way, encode the position of the words without being sequential like RNN? => Positional Encoding

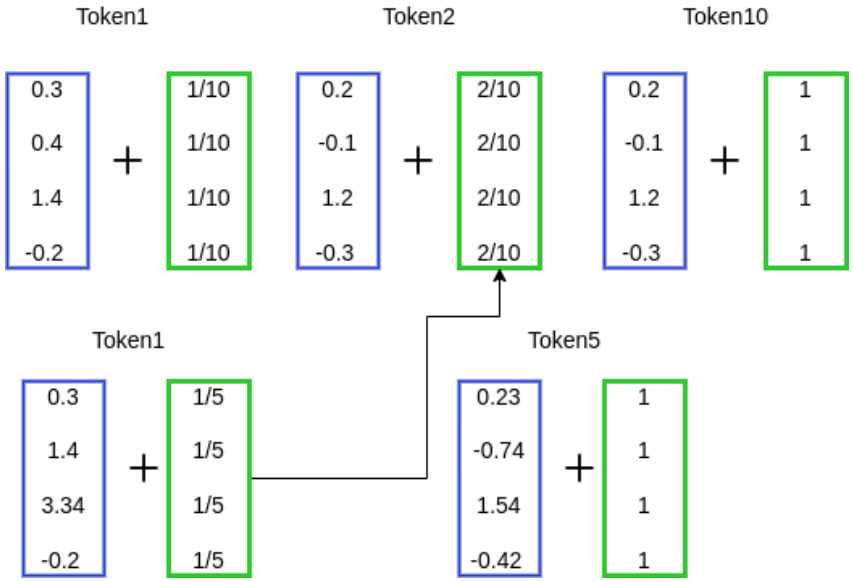


Positional Encoding

Absolute positioning

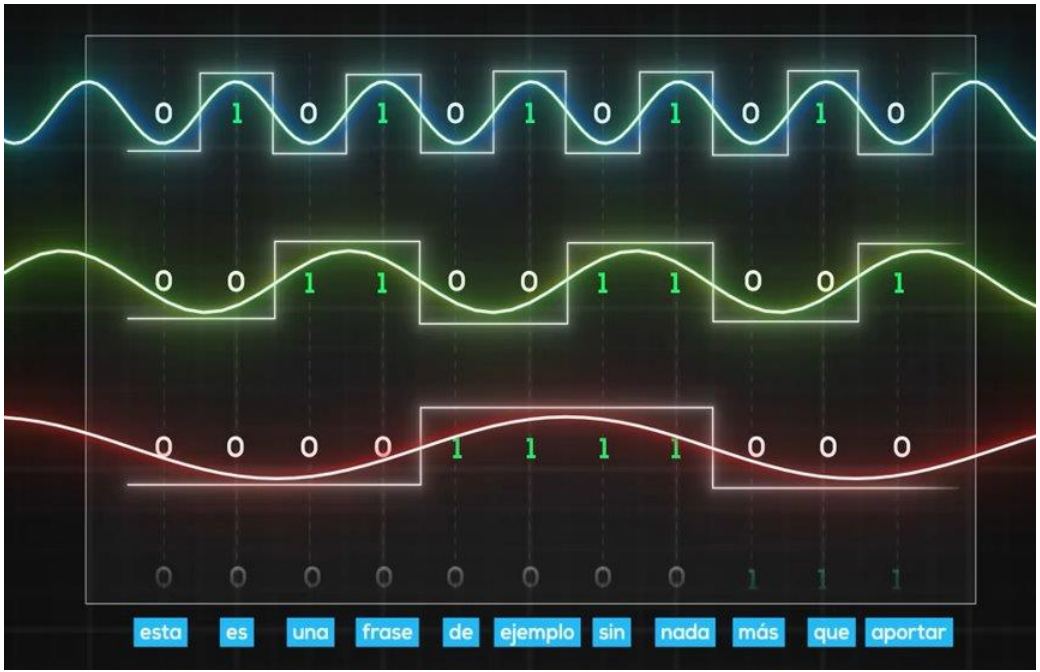


Normalized
absolute positioning



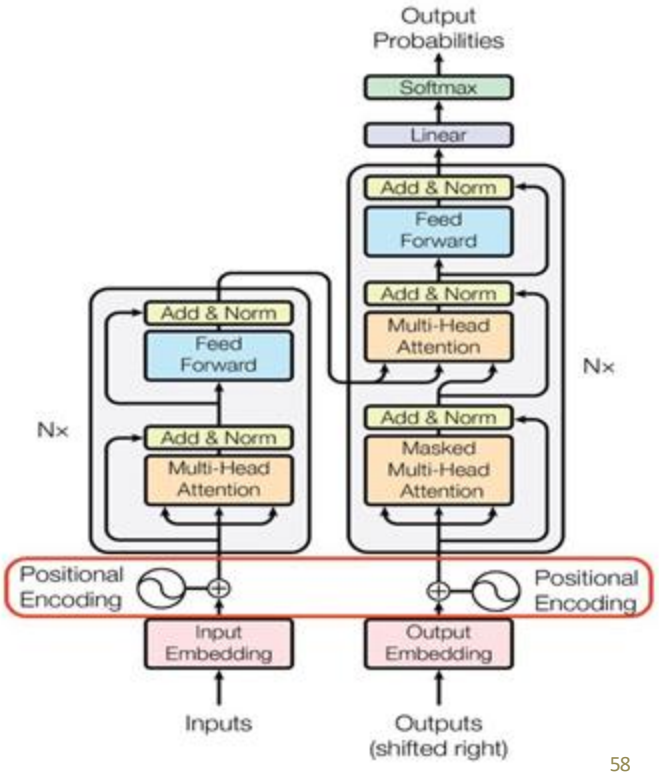
Wave Positional Encoding

0:	0 0 0 0	8:	1 0 0 0
1:	0 0 0 1	9:	1 0 0 1
2:	0 0 1 0	10:	1 0 1 0
3:	0 0 1 1	11:	1 0 1 1
4:	0 1 0 0	12:	1 1 0 0
5:	0 1 0 1	13:	1 1 0 1
6:	0 1 1 0	14:	1 1 1 0
7:	0 1 1 1	15:	1 1 1 1



Transformers: *key concepts*

- New architecture (2017) that allows for the **parallel processing** of data sequences (pixels from images, words from sentences, or sequences of amino acids).
- Input Embedding still maintains the same functionality of converting words into numerical vectors.
- Residual connections that prevent the problem of gradient descent (loss of memory).
- First proposal for translation: x = 6, meaning 6 encoders and 6 decoders.
- Two main concepts: **Attention** and **Positional Encoding.**



Translate example

$$P(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right)$$

$$P(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{1000^{2i/d_{\text{model}}}}\right)$$

i = posición en el vector de embedding
pos = posición en la sentencia
d_model = tamaño vector embedding

$$P = \begin{matrix} \\ I \\ am \\ good \end{matrix} \begin{bmatrix} \sin\left(\frac{\text{pos}}{10000^{0}}\right) & \cos\left(\frac{\text{pos}}{10000^{0}}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^{0}}\right) & \cos\left(\frac{\text{pos}}{10000^{0}}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^{0}}\right) & \cos\left(\frac{\text{pos}}{10000^{0}}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \end{bmatrix}$$

$$P = \begin{matrix} \\ I \\ am \\ good \end{matrix} \begin{bmatrix} \sin(0) & \cos(0) & \sin(0/100) & \cos(0/100) \\ \sin(1) & \cos(1) & \sin(1/100) & \cos(1/100) \\ \sin(2) & \cos(2) & \sin(2/100) & \cos(2/100) \end{bmatrix}$$

Translate example

X =	I	1.769	2.22	3.4	5.8	x_1
	am	7.3	9.9	8.5	7.1	x_2
	good	9.1	7.1	0.85	10.1	x_3

Input matrix
(embedding matrix)

Figure 1.25 – Input matrix

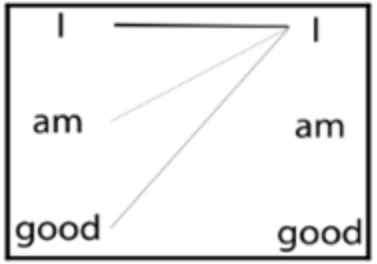
$$X = \begin{bmatrix} 1.769 & 2.22 & 3.4 & 5.8 \\ 7.3 & 9.9 & 8.5 & 7.1 \\ 9.1 & 7.1 & 0.85 & 10.1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0.841 & 0.54 & 0.01 & 0.99 \\ 0.909 & -0.416 & 0.02 & 0.99 \end{bmatrix}$$

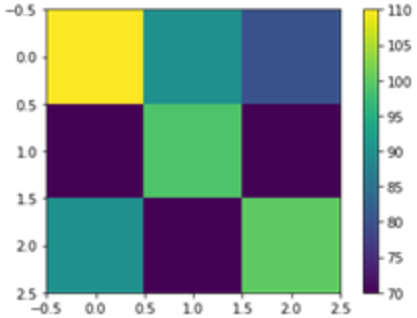
X P

$$= \begin{bmatrix} 1.769 & 3.22 & 3.4 & 6.8 \\ 8.14 & 10.44 & 8.51 & 8.09 \\ 10.0 & 6.68 & 0.87 & 11.09 \end{bmatrix}$$

Figure 1.26 – Adding an input matrix and positional encoding matrix

Translate example





$$
\begin{array}{c} \\ I \\ am \\ good \end{array}
\begin{bmatrix} 3.69 & 7.42 & \dots & 4.44 \\ 11.11 & 7.07 & \dots & 76.7 \\ 99.3 & 3.69 & \dots & 0.85 \end{bmatrix}
\begin{matrix} q_1 \\ q_2 \\ q_3 \end{matrix}
\cdot
\begin{array}{c} I \\ am \\ good \end{array}
\begin{bmatrix} 5.31 & 11.71 & 10.10 \\ 6.78 & 0.86 & 11.44 \\ 0.96 & 11.31 & 5.11 \end{bmatrix}
\begin{matrix} k_1 \\ k_2 \\ k_3 \end{matrix}
=
\begin{array}{c} \\ I \\ am \\ good \end{array}
\begin{array}{c} I \\ \begin{bmatrix} q_1.k_1 \\ q_2.k_1 \\ q_3.k_1 \end{bmatrix} \end{array}
\begin{array}{c} am \\ \begin{bmatrix} q_1.k_2 \\ q_2.k_2 \\ q_3.k_2 \end{bmatrix} \end{array}
\begin{array}{c} good \\ \begin{bmatrix} q_1.k_3 \\ q_2.k_3 \\ q_3.k_3 \end{bmatrix} \end{array}
$$

Q $\qquad$ K^T

$$
QK^T = \begin{array}{c} \\ I \\ am \\ good \end{array}
\begin{array}{c} I \\ \begin{bmatrix} 110 \\ 70 \\ 90 \end{bmatrix} \end{array}
\begin{array}{c} am \\ \begin{bmatrix} 90 \\ 99 \\ 70 \end{bmatrix} \end{array}
\begin{array}{c} good \\ \begin{bmatrix} 80 \\ 70 \\ 100 \end{bmatrix} \end{array}
$$

Matriz de atención = Q x K

Translate example

El mecanismo de atención hace la función de memoria

R = X = sentencia codificada como matriz y su posición

$$X \times W^Q = Q$$
$$X \times W^K = K \quad \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$
$$X \times W^V = V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

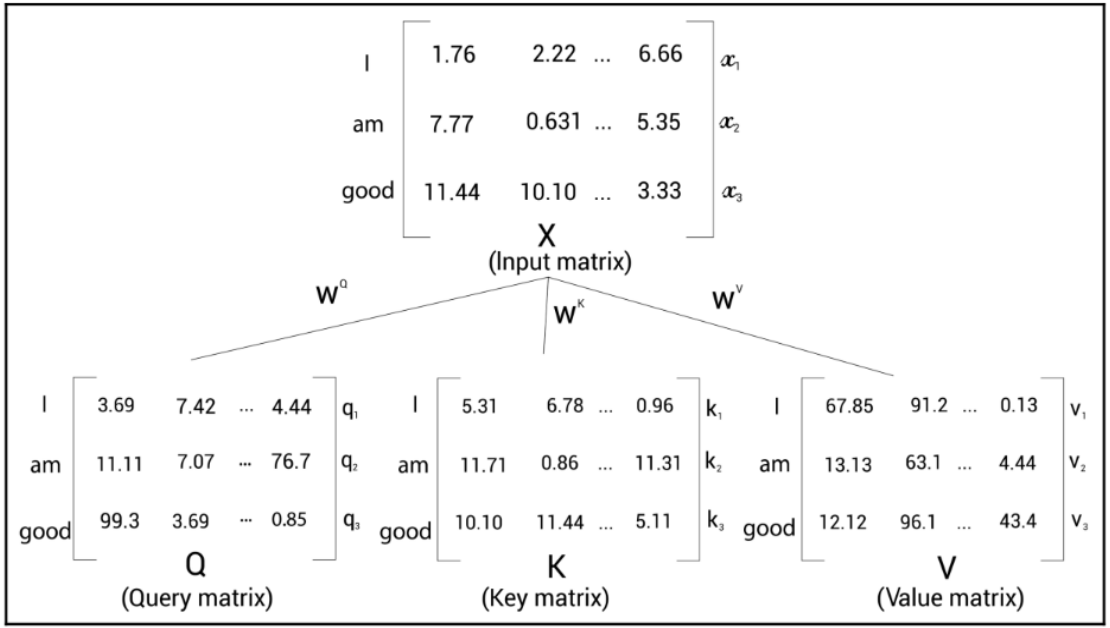
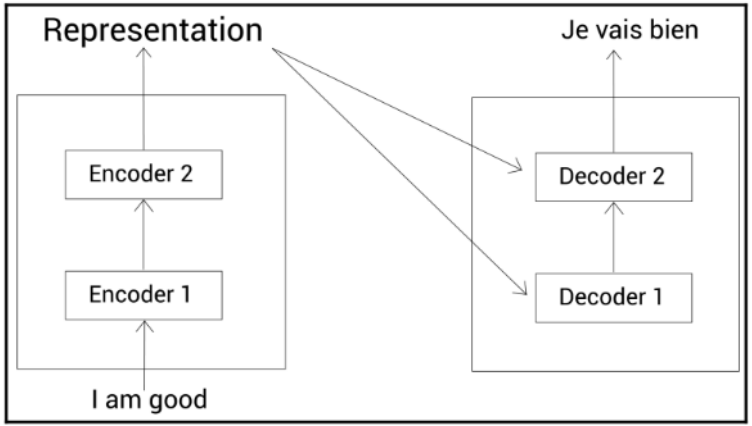
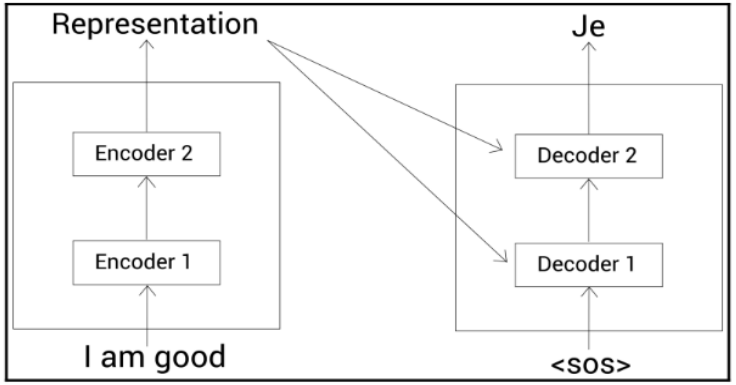


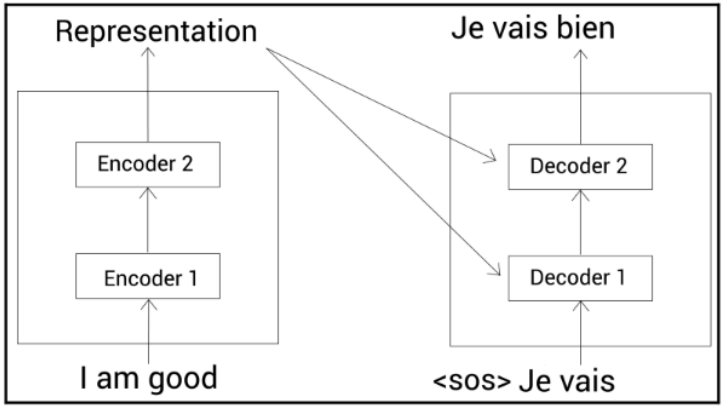
Figure 1.7 – Creating query, key, and value matrices

Translate example



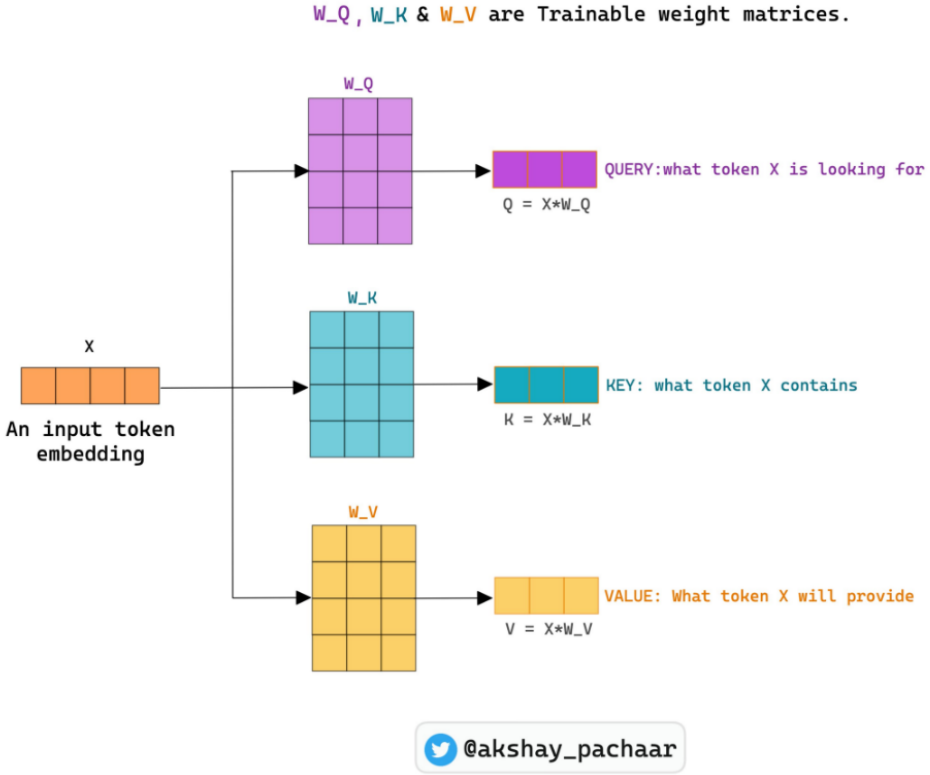


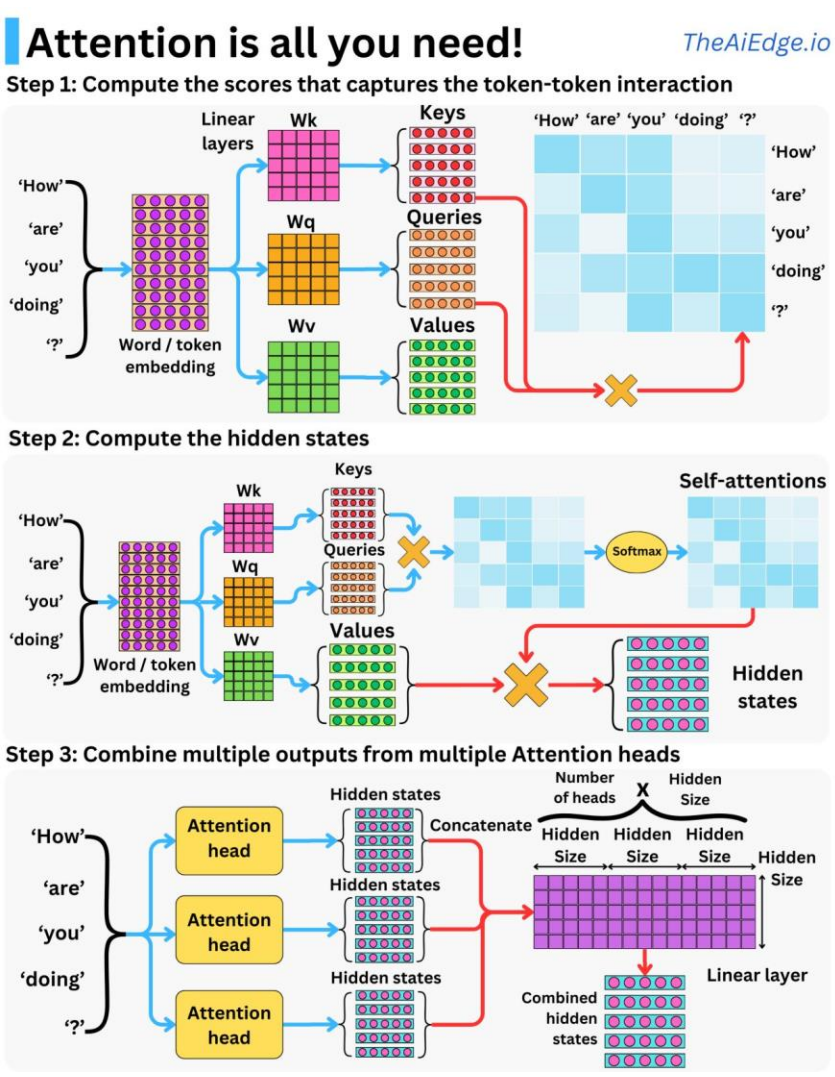
t=0

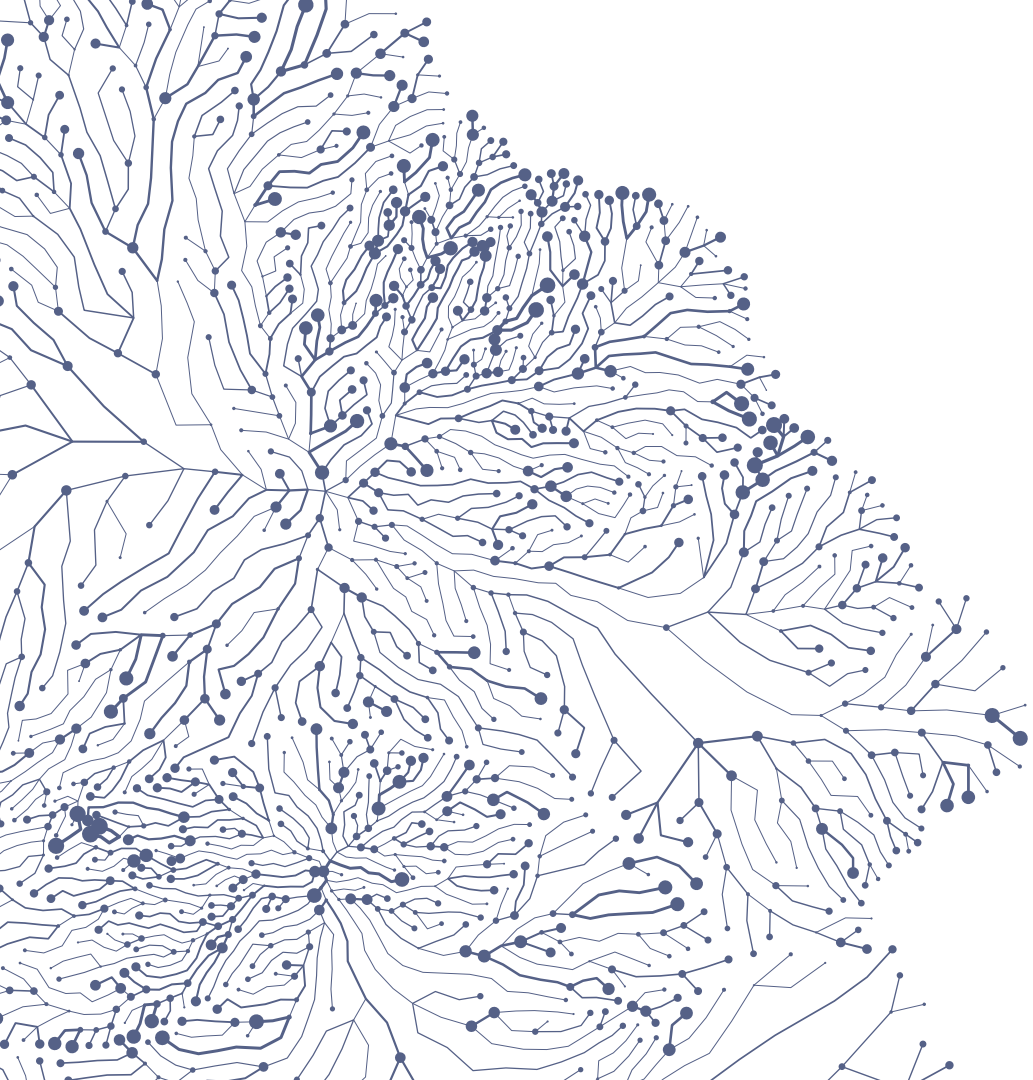


t=2

Transformers Sum up







Foundation Models

Transfer learning

- **LLMs**
  - **Architecture**. E.g., bert-base, bert-large (no data)
  - **Pretraining** focuses on learning a general representation of the language.
    E.g., bert-base-multilingual-uncased, bert-base-multilingual-cased (unlabeled data)
  - **Fine-Tuning** focuses on adapting that representation to a specific task.
    E.g., NER (labelled data)

Pretraining

Masked language modeling

tokens = [Paris, is, a beautiful, city, I, love, Paris]
tokens = [[CLS], Paris, is, a beautiful, [MASK], [SEP], I, love, Paris, [SEP]]

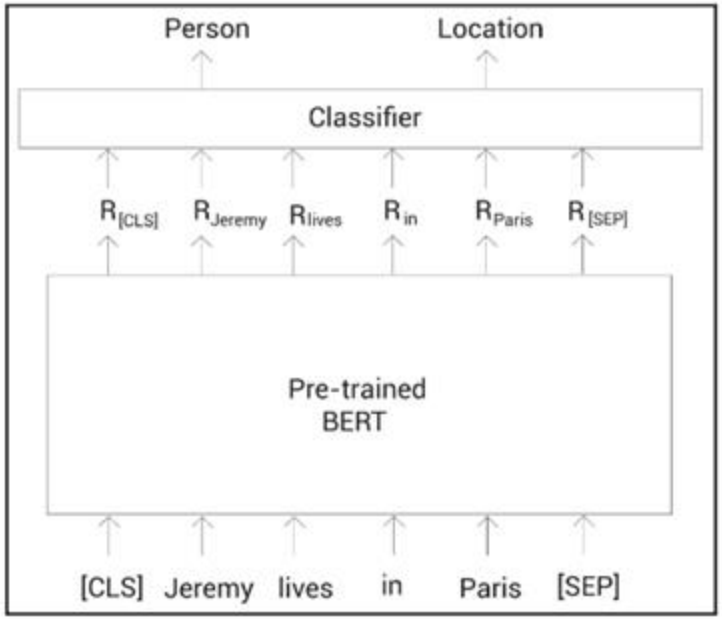
Auto-encoding language modeling

Regla 80-10-10

- 80% se enmascara una palabra (token)
- 10% se reemplaza por un token aleatorio (evitar overfitting)
- 10% se deja el token actual

Existen otras estrategias, como next-sentence-prediction.

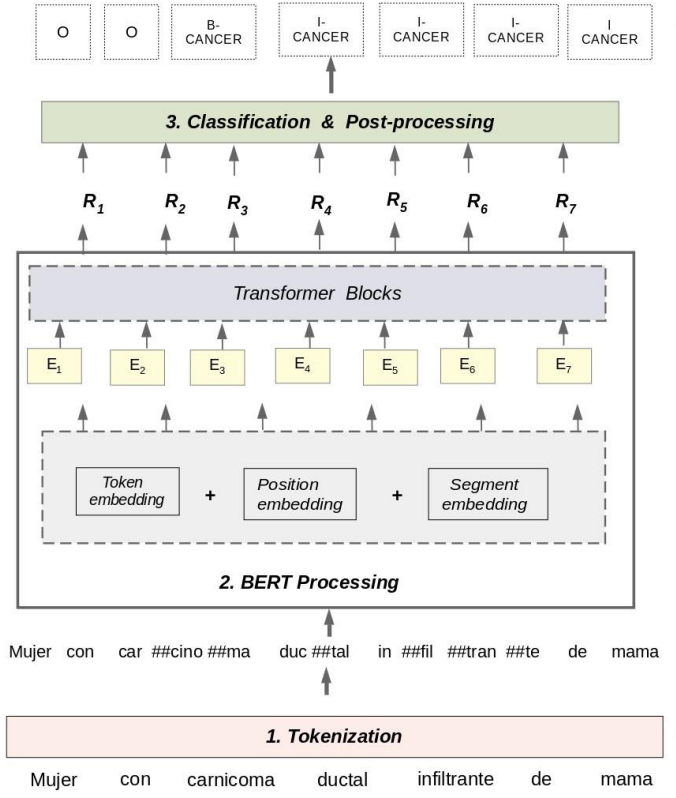
Fine-tuning



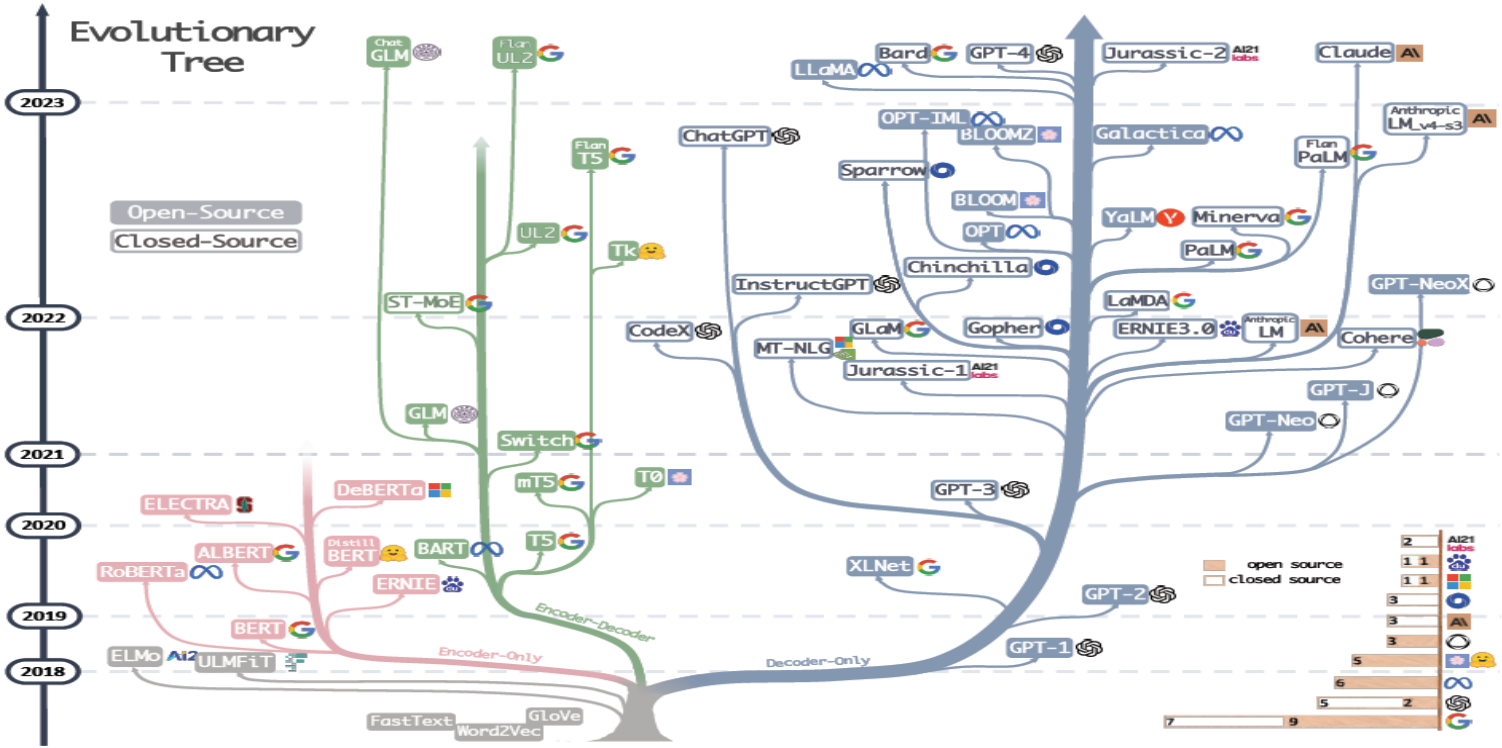
- Language Detection & Translation
- Summaries
- Sentiment analysis
- Named Entity Recognition (NER)
- Extracting Word Relationships
- Entity Linking (Disambiguation)
- Q&A Systems
- Next Phrase Prediction (NSP)

Clinical NER model

- **Architecture:** RoBERTa (no data).
- **Pretraining:** [https://huggingface.co/PlanTL-GOB-ES/roberta-large-bne] Trained with EHR in MareNostrum. This process entails give a sentence of a EHR predict a word of this sentence. (This model only predict words)
- **Fine-Tuning:** we take the pretrained model and with our NER corpus we trained a NER model.



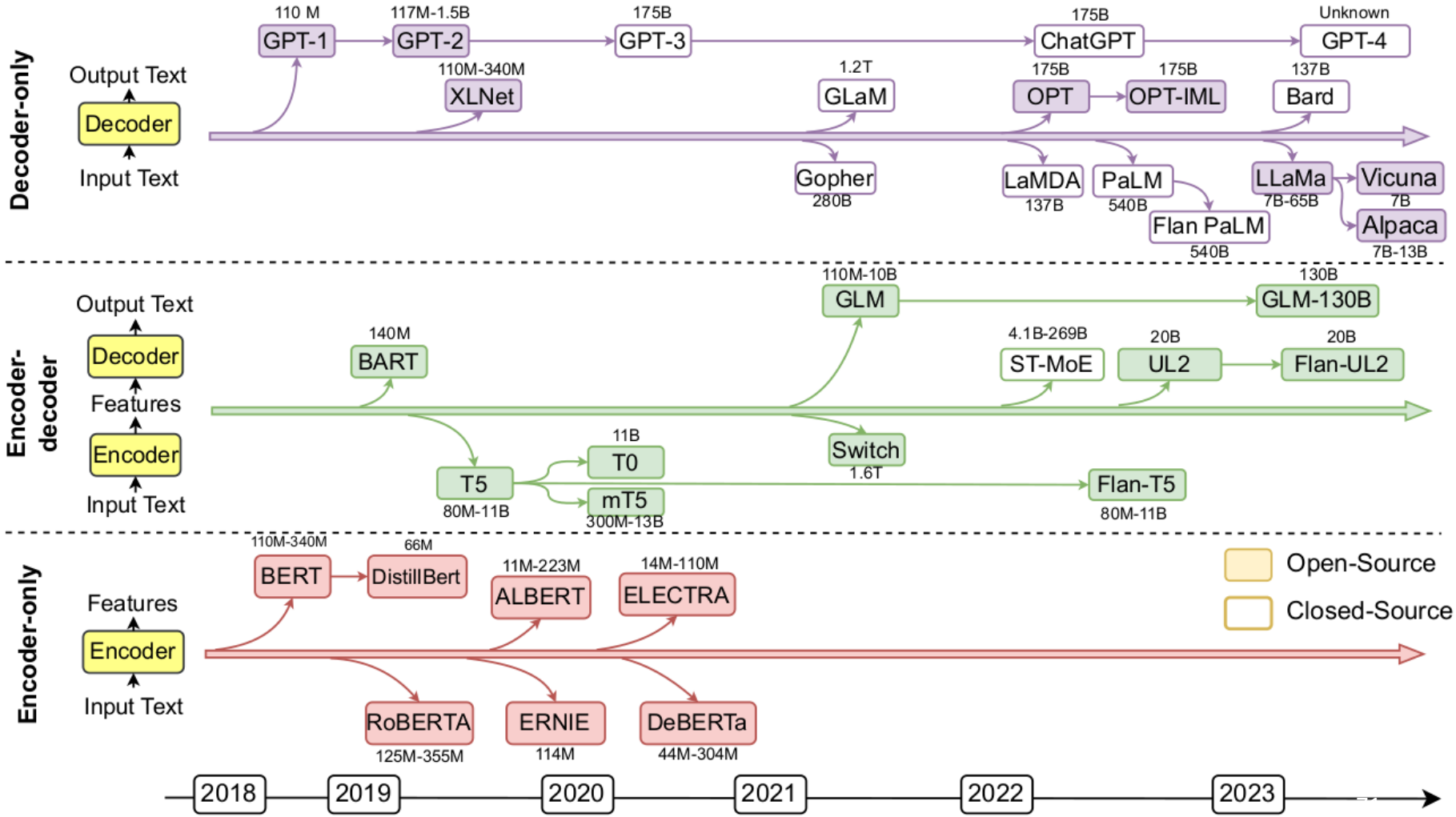
Architecture => Foundation model



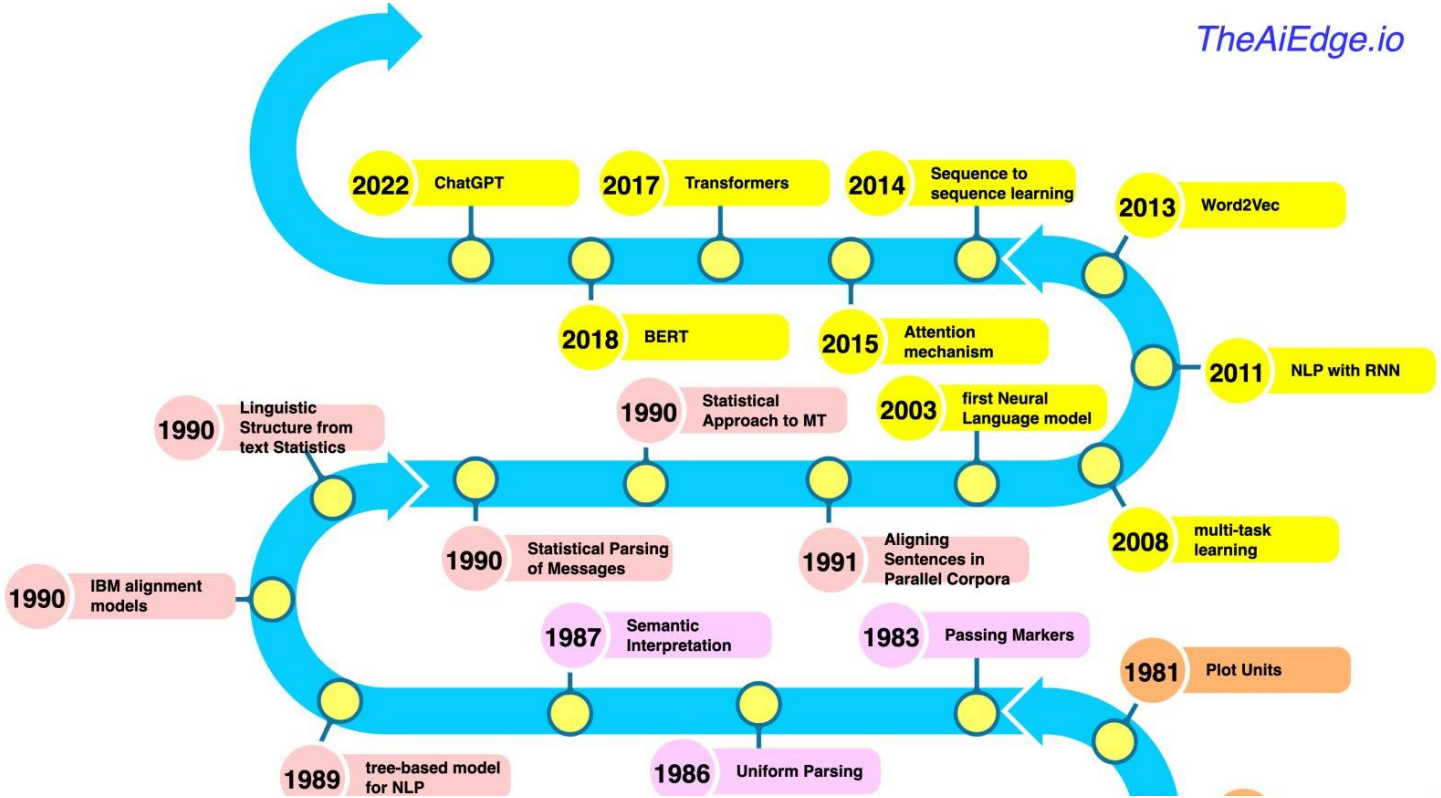
120,385 Agost.
177,097 Nov.
Models
in HF

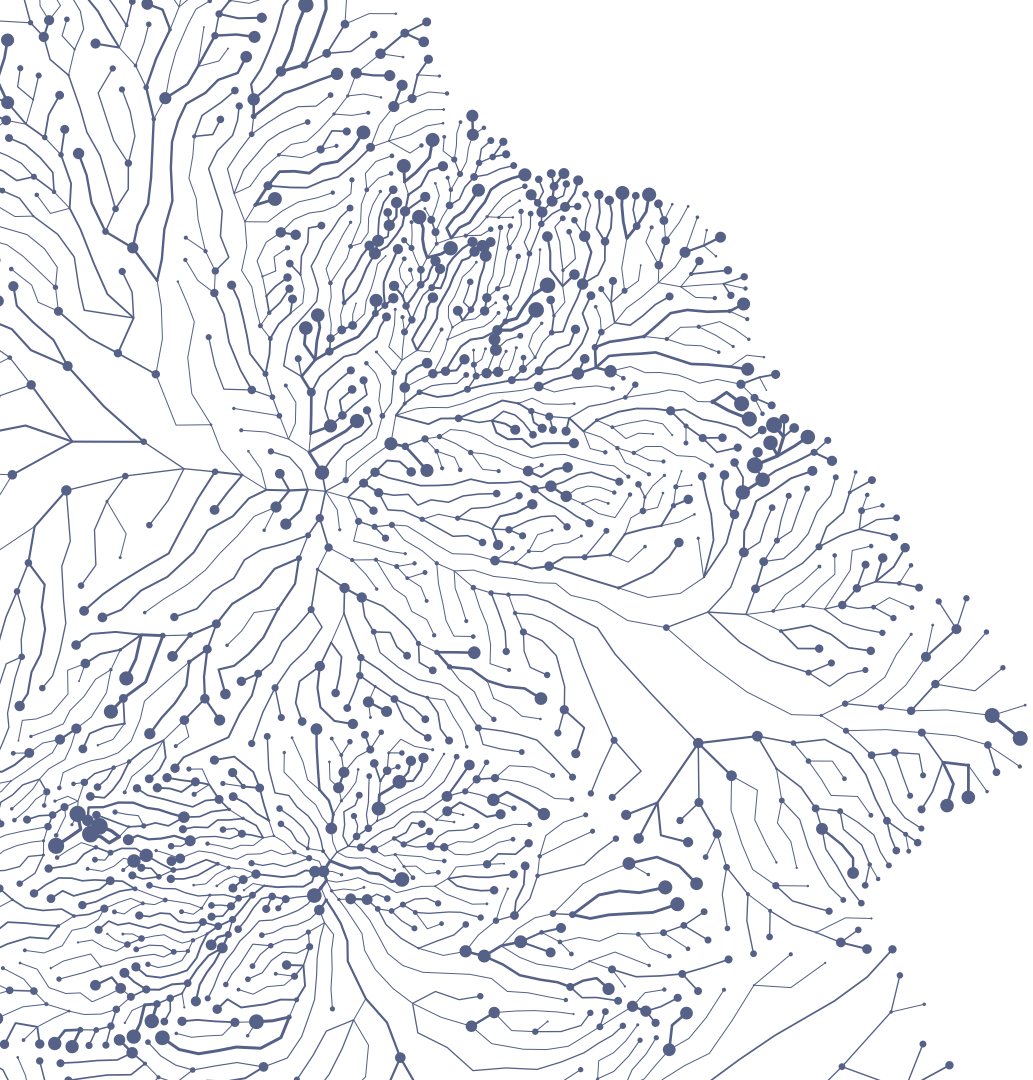
LLAMA 2 (July 2023)
Falcon 180B (September 2023)

Fig. 1: Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond


Decoder-only
Output Text
Decoder
Input Text
110 M
GPT-1
117M-1.5B
GPT-2
175B
GPT-3
175B
ChatGPT
Unknown
GPT-4
110M-340M
XLNet
1.2T
GLaM
175B
OPT
175B
OPT-IML
137B
Bard
Gopher
280B
LaMDA
137B
PaLM
540B
Flan PaLM
540B
LLaMa
7B-65B
Vicuna
7B
Alpaca
7B-13B
Encoder-decoder
Output Text
Decoder
Features
Encoder
Input Text
110M-10B
GLM
130B
GLM-130B
140M
BART
4.1B-269B
ST-MoE
20B
UL2
20B
Flan-UL2
11B
T0
T5
80M-11B
mT5
300M-13B
Switch
1.6T
Flan-T5
80M-11B
Encoder-only
Features
Encoder
Input Text
110M-340M
BERT
66M
DistillBert
11M-223M
ALBERT
14M-110M
ELECTRA
RoBERTA
125M-355M
ERNIE
114M
DeBERTa
44M-304M
Open-Source
Closed-Source
2018
2019
2020
2021
2022
2023

History of NLP

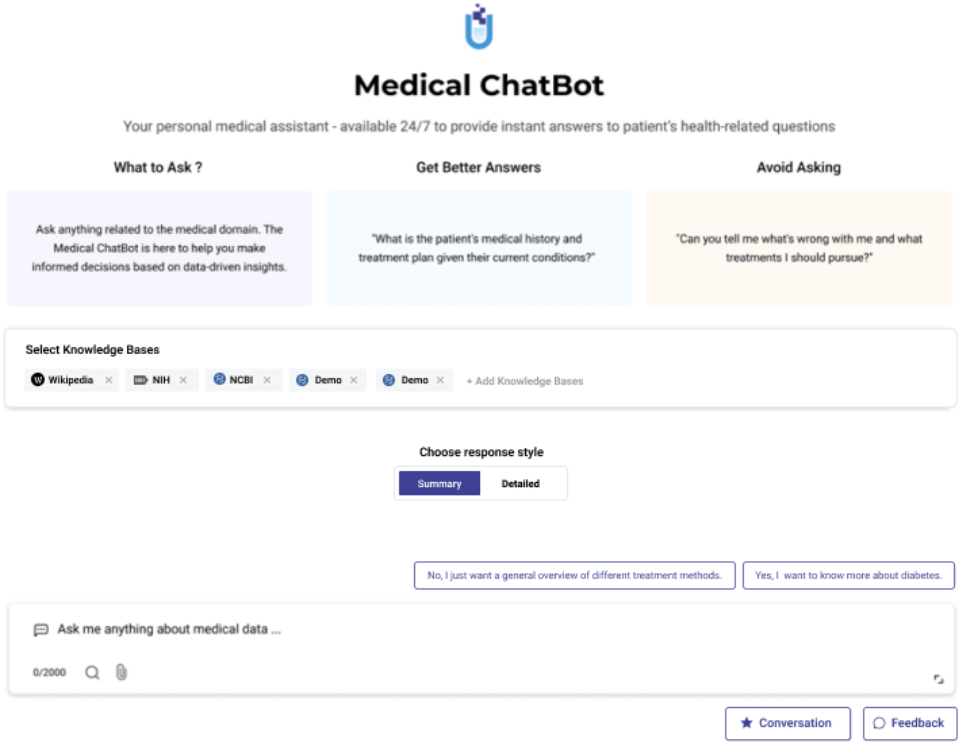




Use cases in medicine

Use cases in medicine

- **Chatbots for Patient Interaction**: AI-powered chatbots can interact with patients, asking questions about their symptoms and providing advice or information. This can be particularly useful for routine inquiries or for providing guidance outside of regular clinic hours. [Example]



Use cases in medicine

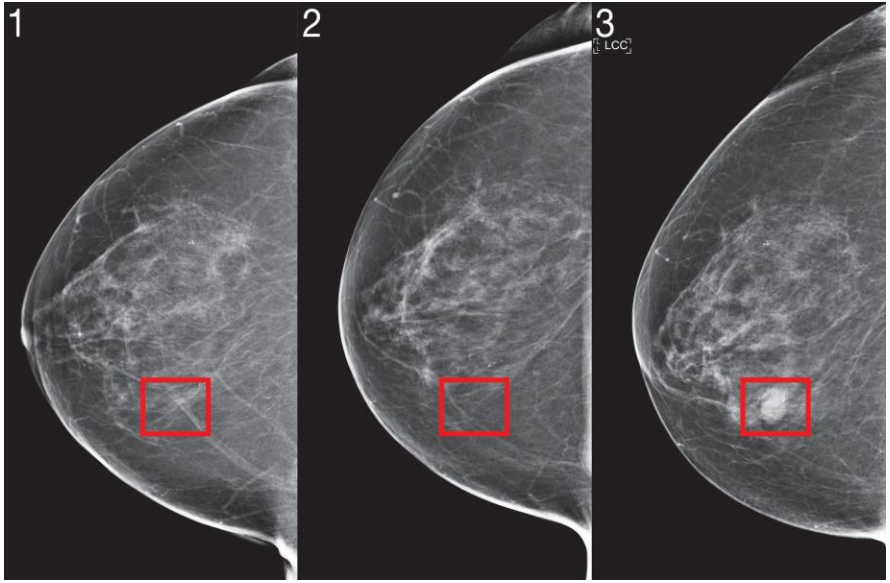
- **Electronical Historical Records Analysis**: AI and NLP systems can analyze vast amounts of medical records to extract critical information. This can include identifying patterns in patient symptoms, treatments, and outcomes. Example: Doctors often write notes in free-form text, and NLP can help extract and structure this information, making it easier to access and analyze.

Use cases in medicine

- **Predictive Analytics**: AI algorithms can predict patient outcomes based on historical data. For example, by analyzing data from past patients, an AI model can predict the likelihood of a patient developing a certain condition or how they might respond to a particular treatment. This helps in personalizing patient care and in making more informed clinical decisions.
- **Drug Discovery and Development**: AI algorithms can analyze scientific data to identify potential new drugs or to repurpose existing drugs for new treatments. This can significantly speed up the drug discovery process and reduce costs.
- **Voice Recognition for Hands-Free Documentation**: NLP and voice recognition technologies enable doctors to dictate notes and have them transcribed automatically, allowing for more efficient documentation and reducing the time spent on paperwork.

Use cases in medicine

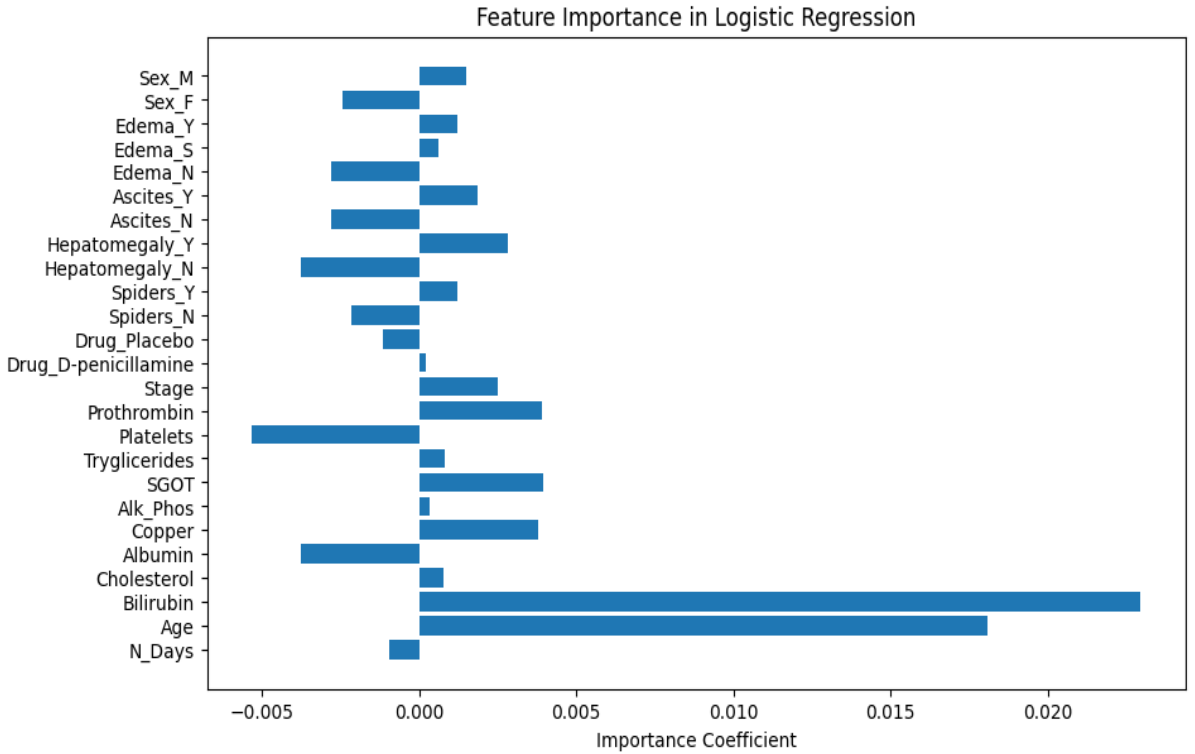
- **Image Analysis and Diagnosis**: AI systems, particularly those employing deep learning, are increasingly being used to analyze medical images such as X-rays, MRI scans, and CT scans. They can help in identifying anomalies and assist radiologists in making diagnoses.



Use cases in medicine

Model interpretability:

- Feature with a coefficient close to 0 indicates that the model does not consider the feature significant for decision-making (*Drug_D-penicillamine*).
- Positive coefficient, as Age, implying that as age increases, the likelihood of the model predicting death also increases.
- A high negative coefficient, as *platelets*, suggests a lower likelihood of being classified as deceased.



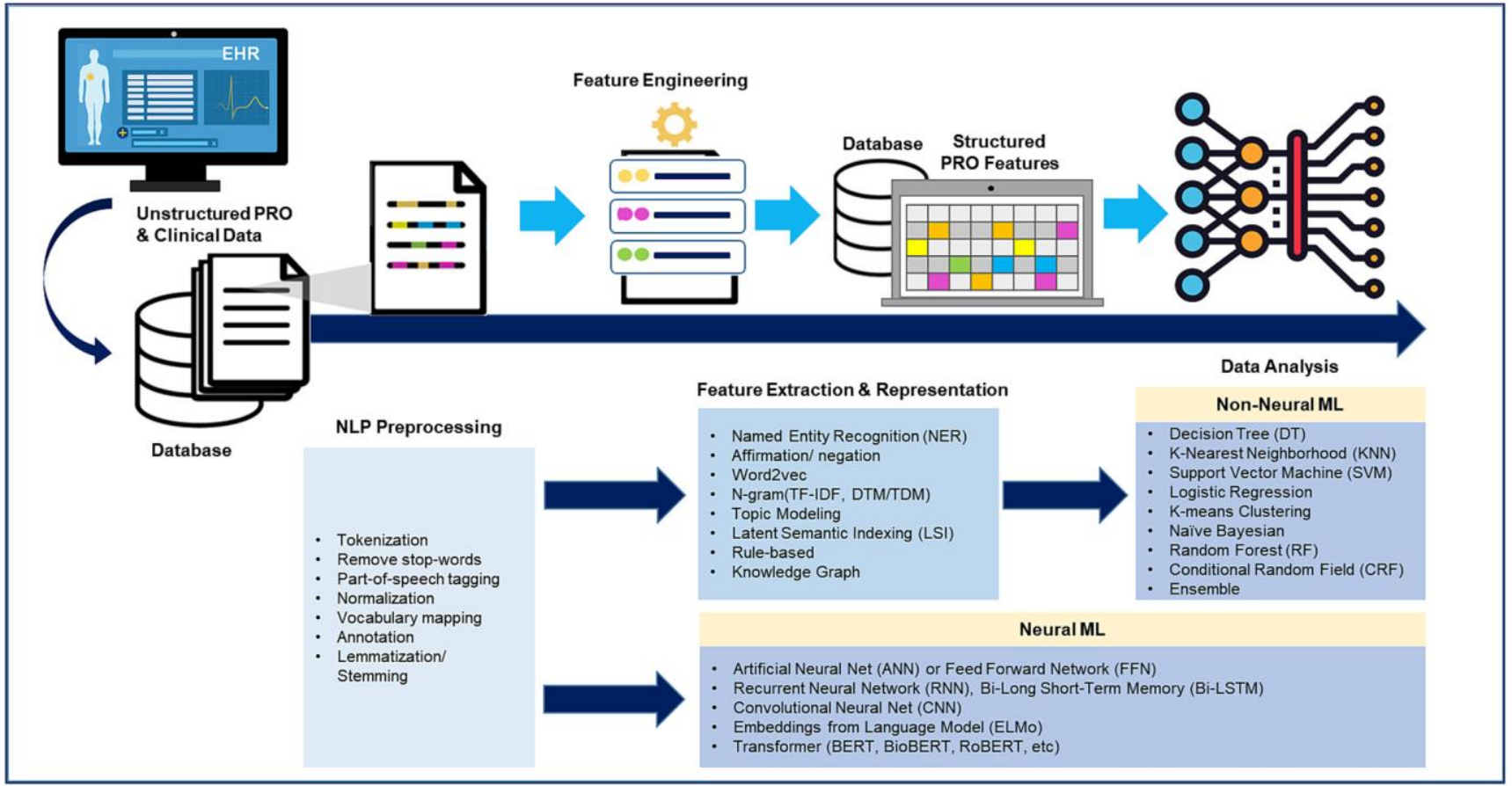


Fig. 2. NLP/ML pipeline for processing unstructured PRO data in EHRs.

Discussion

- Computers do our life easier and improve in a lot of ways, sometimes business take advantage of it with our data.
- They are designate by ourselves and do the things that we want to do. Currently, don’t exist any computer in the word that changes his source code, but the results of ChatGPT are unreal :).
- Computers don’t have moral or ethics, for that reason can't replace humans in work, but the use of them can make betters things.
- A proper use of AI can improve health systems in several ways.

Bibliography

- Resumen de textos literarios en español por medio de modelos lingüísticos Transformers, Guillermo Marco Remón, TFM, UPM.
- Attention is all you need, A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, {. Kaiser, and I. Polosukhin. Advances in Neural Information Processing Systems , page 5998--6008. (2017)
- Getting Started with Google BERT: Build and train state-of-the-art natural language processing models using BERT.
- Transformers for Natural Language Processing: Build, train, and fine-tune deep neural network architectures for NLP with Python, PyTorch, TensorFlow, BERT, and GPT-3.
- García-Barragán, Alvaro, et al. "Structuring Breast Cancer Spanish Electronic Health Records Using Deep Learning." 2023 IEEE 36th International Symposium on Computer-Based Medical Systems (CBMS). IEEE, 2023.
- Solarte-Pabón, Oswaldo, et al. "Transformers for extracting breast cancer information from Spanish clinical narratives." Artificial Intelligence in Medicine 143 (2023): 102625.
- García-Barragán, Alvaro, et al. "Structuring Breast Cancer Spanish Electronic Health Records Using Deep Learning." 2023 IEEE 36th International Symposium on Computer-Based Medical Systems (CBMS). IEEE, 2023.
- Canal de youtube de Carlos Santana Vega, divulgador científico de Inteligencia Artificial.

Thanks!

Álvaro García Barragán
alvaro.gbarragan@upm.es
Center for Biomedical Technology, Technical University, Madrid